

Desarrollo de aplicaciones web con Java

Breve descripción:

El componente proporciona una guía sobre tecnologías y frameworks en el ecosistema Java. Cubre Java con JDBC para la conectividad de bases de datos, Servlets y JSP para el desarrollo web, React JS para interfaces de usuario, Hibernate para ORM, y Spring Framework para aplicaciones empresariales, destacando la inyección de dependencias y la configuración simplificada con Spring Boot.

Julio 2024

Tabla de contenido

Introducción	5
1. Java con JDBC.....	7
1.1. Clases e interfaces Java	8
1.2. Conexión a base de datos desde aplicación Java.....	10
2. Servlets y JSP	14
2.1. Introducción a servlets	17
Ciclo de vida de un Servlet	19
Ejemplo de Servlet	20
2.2. Formularios HTML con Servlets.....	23
2.3. HTTP Session	25
2.4. Elementos de JSP.....	28
2.5. MVC.....	31
3. Fundamentos de React JS	33
¿Qué es React JS?	34
¿Qué es JSX?	34
3.1. Instalando React JS.....	34
3.2. Crear un proyecto.....	43
3.3. Propiedades y estados de los componentes	48

Eventos en React	50
Listas y Keys	51
Formularios	53
Concepto de clases en React	54
Funciones Hooks.....	55
Ciclos de vida de los componentes de React	56
Ejecutar la aplicación.....	61
4. Framework Hibernate	64
4.1. Operaciones básicas	70
Método Save	72
Método Get	75
Método CreateQuery	78
Actualizaciones	82
Método Delete	86
4.2. Asociaciones.....	91
Relaciones OneToOne	91
Relaciones OneToMany - ManyToOne	93
Relaciones ManyToMany	94
4.3. Ciclo de vida	95

5. Framework Spring.....	98
5.1. Introducción	100
Módulos del Framework	103
Creación de la aplicación	106
5.2. Spring Data	112
Módulos de Spring Data	114
5.3. Spring Boot y MySQL.....	116
Contenedor de aplicaciones integrado.....	117
Starters	117
5.4. Validaciones.....	127
Síntesis	130
Material complementario	131
Glosario	132
Referencias bibliográficas	134
Créditos	135

Introducción

En el ámbito del desarrollo de software, Java se ha consolidado como uno de los lenguajes de programación más robustos y versátiles. Este documento proporciona una guía comprensiva sobre algunas de las tecnologías y frameworks más importantes utilizados en el ecosistema Java, con un enfoque particular en la conectividad de bases de datos, el desarrollo web y la gestión de la persistencia de datos.

El contenido de este documento se estructura en cinco secciones principales:

- **Java con JDBC**

Se examina el uso de Java Database Connectivity (JDBC), una API fundamental para conectar aplicaciones Java con bases de datos relacionales. Se describen las clases e interfaces necesarias, los pasos para establecer una conexión y se proporciona información sobre los drivers JDBC para distintos sistemas de gestión de bases de datos (SGBD).

- **Servlets y JSP**

Este tema introduce los Servlets y JavaServer Pages (JSP), tecnologías clave para el desarrollo web en Java. Se abordan su arquitectura, ciclo de vida, y la forma de manejar formularios HTML y sesiones HTTP, así como la implementación del patrón de diseño Modelo-Vista-Controlador (MVC).

- **Fundamentos de React JS**

Aquí se exploran los fundamentos de React JS, una biblioteca de JavaScript desarrollada por Facebook para construir interfaces de usuario. Se incluye la instalación, creación de proyectos, manejo de componentes y estado,

eventos, listas y keys, formularios, ciclo de vida de los componentes y el uso de hooks.

- **Hibernate**

Hibernate es un framework de mapeo objeto-relacional (ORM) que facilita la interacción con bases de datos en aplicaciones Java. Esta sección cubre la configuración de Hibernate, el uso de clases de entidad y anotaciones, y las operaciones básicas como guardar, obtener, actualizar y eliminar registros, así como las asociaciones entre entidades y el ciclo de vida de los objetos.

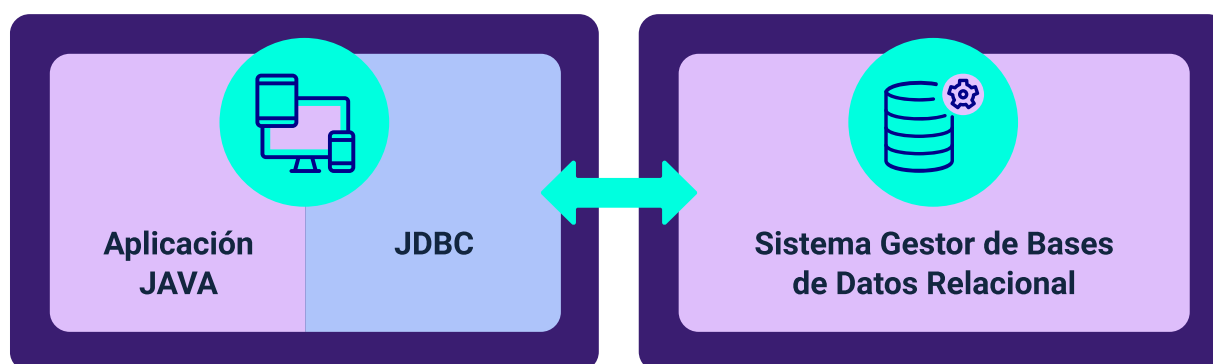
- **Spring**

Se presenta el Spring Framework, una herramienta poderosa para el desarrollo de aplicaciones empresariales en Java. Se destacan la inyección de dependencias (DI), el principio de inversión de control (IOC), y se examinan módulos importantes como Spring Data y Spring Boot. Además, se discuten las validaciones en Spring Boot y se proporciona un ejemplo de mapeo con JPA.

1. Java con JDBC

Cuando se habla de JDBC (Java Database Connectivity), se hace referencia a un driver que encapsula un conjunto de clases e interfaces escritas en Java, creadas para establecer conexiones, enviar sentencias SQL y procesar los datos resultantes sobre bases de datos relacionales.

Figura 1. Java con JDBC



En términos más generales, JDBC es uno de los puentes que puede ser utilizado en Java para comunicar una aplicación Java con un servidor de bases de datos relacionales. Trabajar con conexiones a bases de datos mediante JDBC implica realizar cuidadosamente las siguientes acciones:

- Manejar manualmente las conexiones y desconexiones a la base de datos.
- Ser cuidadosos en la escritura de sentencias SQL a ser ejecutadas en el servidor.
- Conocer las relaciones existentes en el modelo de base de datos a manipular.
- Conocer las características de la base de datos a manipular, ya que esto podría afectar las consultas.

- **Tenga presente**

Cada sistema gestor de base de datos (SGBD) provee su propio JDBC para la manipulación de sus sistemas con la aplicación Java. Dependiendo de la base de datos a la que se desee conectar, se debe descargar el driver JDBC correspondiente.

La siguiente tabla muestra las direcciones oficiales para la descarga del driver JDBC para las bases de datos relacionales más comúnmente utilizadas en la industria del software.

Tabla 1. Drivers JDBC para SGBD

Base de Datos	Página oficial para descarga de JDBC
Mysql	https://dev.mysql.com/downloads/connector/j/
SQL Server	https://docs.microsoft.com/en-us/sql/connect/jdbc/download-microsoft-jdbc-driver-for-sql-server?view=sql-server-ver15
Oracle	https://www.oracle.com/database/technologies/jdbc-upc-downloads.html
PostgreSQL	https://jdbc.postgresql.org/download/
DB2	https://www.ibm.com/support/pages/db2-jdbc-driver-versions-and-downloads

1.1. Clases e interfaces Java

Además de disponer del driver JDBC correspondiente para la base de datos que se desea conectar, se requiere un conjunto adicional de clases e interfaces Java para hacer uso del driver. Para ello, es necesario importar los paquetes java.sql o javax.sql. A continuación, se describen algunas de las clases más importantes dentro de estas

librerías, y otros elementos, para gestionar correctamente una base de datos por medio de JDBC.

- **DriverManager**

Clase estática que permite la gestión del driver JDBC disponible para la aplicación. Por medio del método `getConnection`, crea una instancia de tipo `Connection` mediante la cual se realiza el flujo de envío y recibo de información hacia y desde la base de datos destino.

- **Connection**

Interfaz Java que permite establecer una conexión con una base de datos específica por medio de la cual se puede extraer información de la descripción de tablas, procedimientos almacenados y ejecutar sentencias SQL. Se instancia un objeto de este tipo invocando el método `getConnection` del `Driver Manager`, al cual se le deben pasar como parámetros las credenciales (usuario y contraseña) del usuario de la base de datos destino y la cadena de conexión correspondiente.

- **Statement**

Interfaz Java usada para la ejecución de sentencias SQL estáticas y retornar los resultados que son producidos por estas sentencias. Una instancia de este tipo se obtiene de la ejecución del método `createStatement()` desde una instancia de un objeto de tipo `Connection`.

- **ResultSet**

Tipo especial de dato que representa un conjunto de datos de tipo tabular, ideal para referenciar resultados obtenidos por medio de una consulta que retorna un conjunto de datos estructurados, como, por ejemplo, los de una

consulta SELECT. Un ResultSet es un apuntador, tipo cursor, direccionado a la primera posición de toda la estructura tabular que está referenciando.

- **El método next()**

Además de desplazarse una posición de forma lineal dentro de la estructura, también devuelve como resultado un valor booleano para informar si el desplazamiento realizado encontró, o no, un conjunto de datos válidos. Así es posible determinar cuándo se alcanza el final de los elementos.

Clases e interfaces Java

Para profundizar y conocer los elementos más importantes de las clases e interfaces Java más usadas, lo invitamos a consultar el PDF "**Clases e interfaces Java**", el cual se encuentra en la carpeta Anexos.

1.2. Conexión a base de datos desde aplicación Java

Garantizar la conexión a una base de datos requiere la aplicación concreta de los pasos ya explicados.

Los pasos que deben realizarse para garantizar una conexión a base de datos desde una aplicación Java, utilizando JDBC, son:

- a) Descarga del driver JDBC (Si no se tiene).
- b) Importar el driver descargado en las librerías del proyecto JAVA.
- c) Importar el paquete `java.sql.*`.
- d) Inicializar el driver usando la sentencia `Class.forName`.
- e) Crear un Objeto de tipo `Connection` por medio del `DriverManager`.

- f) Crear un objeto Statement que permite configurar las sentencias SQL desde el Objeto de Connection creado en el punto anterior.
- g) Ejecutar la sentencia SQL usando cualquiera de los métodos del objeto Statement.
- h) Procesar los resultados de la sentencia ejecutada por medio de un ResultSet y sus métodos de apoyo.

Para afianzar los conocimientos logrados hasta este punto, se recomienda revisar el video propuesto a continuación. En este, se presenta un ejemplo sencillo y se describen las prácticas de cada paso descrito.

Video 1. Conexión JDBC



[Enlace de reproducción del video](#)

Síntesis del video: Conexión JDBC

En este video se explica cómo conectar una aplicación Java a un sistema gestor de bases de datos utilizando el driver JDBC. El ejemplo se desarrolla utilizando un servidor local de bases de datos MySQL, instalado a través del paquete XAMPP, y el entorno de desarrollo integrado (IDE) NetBeans.

Primero, se descarga el driver correspondiente para MySQL y se configura en el proyecto de NetBeans. Se menciona la importancia de contar con tres elementos clave: una cuenta de usuario y contraseña en el servidor, y la URL de conexión. La URL, en este caso, es localhost con el puerto por defecto 3306 y la base de datos llamada "prueba".

Se crea un proyecto en NetBeans y se configura para conectarse a la base de datos MySQL. La tabla utilizada en el ejemplo se llama "usuario" y tiene tres columnas: user_id, username y user_password. Para establecer la conexión, se agrega el driver descargado al proyecto y se instancian las librerías necesarias.

A continuación, se define la cadena de conexión con los parámetros de usuario, contraseña y URL del servicio. Se importan las librerías necesarias en Java, como java.sql, que permiten la creación de objetos de tipo Connection, Statement y ResultSet.

Luego, se instancia el driver con la sentencia Class.forName() y se maneja cualquier excepción con un bloque try-catch. Se establece la conexión con el DriverManager y se crea un objeto Statement para ejecutar consultas SQL.

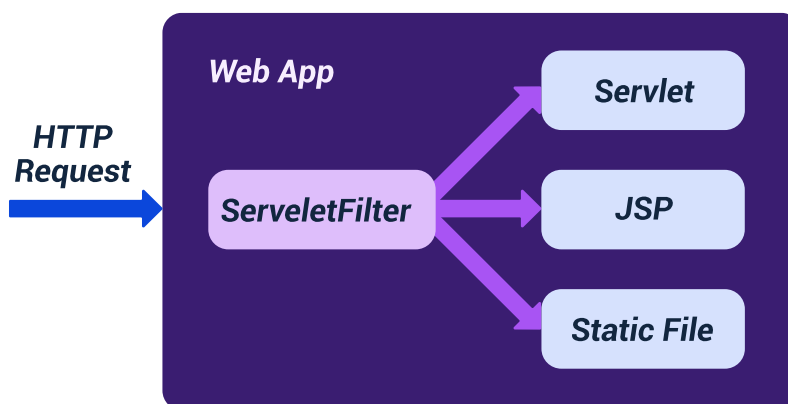
El video describe cómo insertar datos en la tabla "usuario" y cómo verificar la inserción mediante una consulta SELECT. Se utiliza un ciclo para procesar y mostrar los resultados de la consulta.

Finalmente, se realiza la conexión a la base de datos MySQL utilizando el driver JDBC y se destacan las funcionalidades básicas de inserción y consulta de datos.

2. Servlets y JSP

Los Servlets y los JSP (Java Server Pages), son tecnologías que se utilizan para realizar ejecuciones en servidores Java, permitiendo extender los servicios de un servidor web. (Berzal, 2007)

Figura 2. Java con JDBC



Un ejemplo sencillo de lo que se puede hacer con JSP y Servlet es que, por medio de una página JSP, se puede crear un formulario que contenga una serie de datos como: primer nombre, segundo nombre, edad y correo. Luego, a través de un servlet, se realiza el envío de estos registros para que sean almacenados en una base de datos y, finalmente, el usuario pueda acceder a esta información.

Conozca, a continuación, los procesos principales de estas tecnologías y algunos aspectos generales del uso de JSP y Servlet:

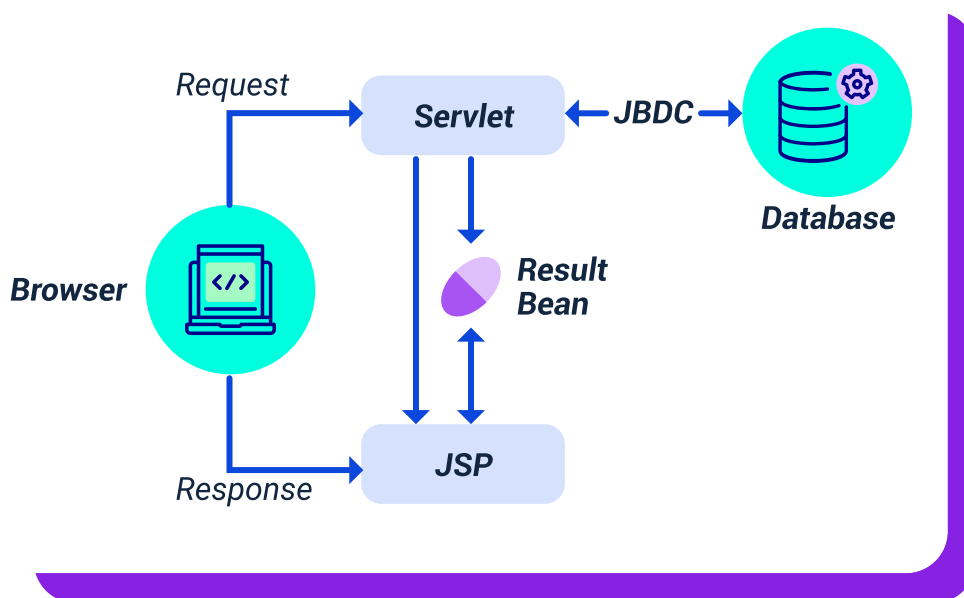
- **Los servlets**

Adecuan el proceso de las peticiones que entran al servidor, procesan los datos de formularios, generan contenidos de varios tipos y redireccionan las peticiones.

- **JSP (JavaServer Pages)**

Es un código Java que se ejecuta en un servidor remoto. Lee las acciones del usuario desde formularios HTML, donde el servidor realiza consultas, usualmente a bases de datos, para obtener la información solicitada y devolver una página HTML dinámica y en tiempo real.

Figura 3. Arquitectura web que involucra JSP y Servlets



- **Los archivos JSP**

Están formados por código Java dentro de etiquetas <html> con extensión .jsp. Estos archivos deben ser guardados en la carpeta del contenido web del servidor. El código JSP inicia con <% %>.

- **Ejemplo**

Este es un ejemplo sencillo de código JSP.

```
<%@ page language='java' contentType='text/html; charset=iso-8859-1'%>
```

```
<%@ page import='java.util.Date' %>
```

```
<html>
```

```
<head>
```

```
    <title>Hola Planeta</title>
```

```
</head>
```

```
<body>
```

```
    <p>Bienvenido a la página JSP. </p>
```

```
    <p>La hora del servidor es <%= new Date() %> </p>
```

```
    <% out.println("HOLA PLANETA"); %>
```

```
</body>
```

```
</html>
```

- **Software requerido**

El software necesario para trabajar con JSP y servlets es: Java JEE IDE (NetBeans, Eclipse) y un servidor web (Tomcat).

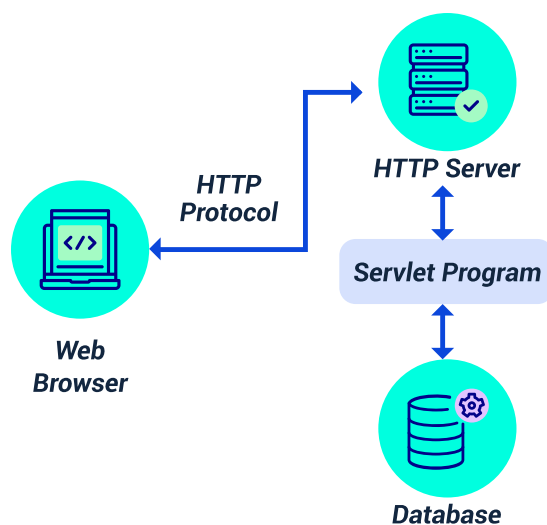
Figura 4. Entorno de desarrollo Eclipse IDE



2.1. Introducción a servlets

Servlet es un programa que se lleva a cabo en una aplicación o servidor web; se emplea como una capa que intercede entre la base de datos y el servidor de peticiones HTTP, por medio de un navegador web o cualquier otro cliente HTTP (Gómez, 2017).

Figura 5. Arquitectura de un servlet



A continuación, se presentan algunas ventajas de utilizar servlets:

- Rendimiento: a diferencia de CGI (Common Gateway Interface), el rendimiento de un servlet es mucho mejor.
- No necesita procesos independientes para operar las solicitudes del cliente.
- Dado que los servlets están escritos en Java, son independientes de la plataforma.
- Fiabilidad: resguardan los recursos del servidor gracias a la seguridad proporcionada por Java.

Las tareas principales de un servlet son:

- **Datos de transmisión**

Se encarga de leer claramente los datos de transmisión de un cliente (navegador), incluyendo un formulario HTML.

- **Remisión de peticiones HTTP**

Remite una petición HTTP de los datos contenidos en el navegador, incluyendo cookies.

- **Tratamiento de datos**

Trata los datos y produce resultados accediendo a la base de datos.

- **Envío de documentos**

Envía documentos al cliente (navegador), los cuales pueden ser de varios tipos como textos, binarios, etc.

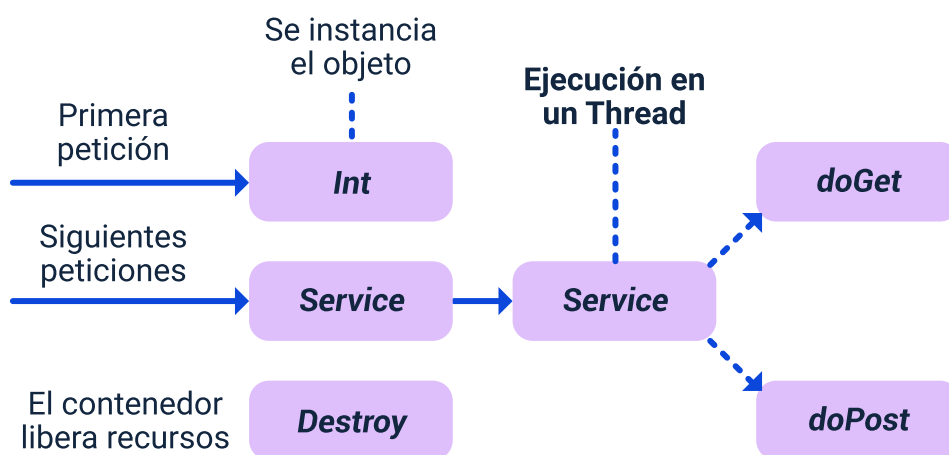
- **Envíos de respuesta**

Realiza envíos de respuesta HTTP implícita al cliente (navegador).

Ciclo de vida de un Servlet

A diferencia de las clases en Java, que inician con un método `main()`, en los servlets se invocan ciertos métodos al recibir peticiones. A esta metodología se le llama ciclo de vida de un servlet y se compone de tres métodos: `init`, `service` y `destroy`.

Figura 6. Ciclo de vida de un Servlets



Conocer el ciclo de vida de un Servlet; procurar llevar registro, en la libreta personal de apuntes, de los aspectos más importantes y destacados.

- **Init**

El servlet inicia por medio de una llamada al método `init()`. Este método está definido para ser llamado una sola vez. Cuando el servlet es invocado, se crea una instancia del servlet, generando un hilo nuevo para cada solicitud.

- **Service**

En segundo lugar, es llamado el método de servicio para gestionar la solicitud del cliente. La función del método `service` es atender las

peticiones GET y POST de los clientes; cuando se llama a este método, invoca a `doGet()` y `doPost()`, dependiendo del caso.

- **Destroy**

Se llama al método `destroy` para finalizar el servlet. Como su nombre lo indica, se encarga de destruir el servlet sobrescribiendo el método `destroy()`.

Ejemplo de Servlet

El siguiente código presenta un "Hola Mundo" en un servlet, con sus métodos de ciclo de vida.

```
public class HolaMundo extends HttpServlet {  
  
    private String mensaje;  
  
    public void init () throws ServletException  
  
        { // inicia la asignación de mensaje hola mundo  
  
            mensaje = "Hola Mundo";  
  
        }  
  
    public void doGet (HttpServletRequest request,HttpServletResponse response)  
        throws ServletException, IOException  
  
        { // Se especifica el formato de la respuesta utilizando método getWriter  
  
            salida = response.getWriter ();  
  
            // Genera respuesta de la petición
```

```
        salida.println("<h1>" + mensaje + "</ h1>");  
    }  
  
    public void destroy ()  
  
    {.
```

En el código anterior, se hace lo siguiente:

- a) Se crea la clase HolaMundoServlet que hereda de HttpServlet.
- b) La clase HttpServlet contiene el método doGet que se encarga de recibir la petición del usuario desde el navegador.
- c) El método doGet recibe dos argumentos: request (para procesar la petición) y response (para dar respuesta a la petición).
- d) Se sobrescribe el método doGet para crear la lógica de respuesta al usuario, se establece el formato de la respuesta en HTML y se establece el contenido de la respuesta.

Explore el vídeo que se propone para ahondar en los aspectos más importantes de Servlet.

Video 2. Servlet



Enlace de reproducción del video

Síntesis del video: Servlet

El video explica cómo crear un primer Servlet utilizando NetBeans y un servidor web Tomcat. Comienza con la preparación del entorno, que incluye la instalación de NetBeans o Eclipse y la configuración de Tomcat. Luego, se procede a crear un nuevo proyecto de aplicación web en NetBeans, configurando el servidor y seleccionando la versión adecuada de Java EE. Dentro del proyecto, se genera una estructura que incluye un archivo index.html y se crea un paquete para contener el Servlet. Se configura el descriptor web (web.xml) para mapear el Servlet y se asignan un nombre y una URL.

La clase del Servlet extiende `HttpServlet` e implementa los métodos `doGet` y `doPost` para manejar las solicitudes HTTP. El método `doGet` se configura para responder con un mensaje HTML simple. Una vez que se verifica la configuración en `web.xml`, se ejecuta el proyecto y se asegura de que el servidor web esté funcionando. Al acceder a la URL configurada, el Servlet responde con un mensaje "Hola mundo". El video concluye con la explicación de este proceso, destacando la importancia de la configuración correcta y la verificación del servidor antes de la ejecución.

2.2. Formularios HTML con Servlets

Las páginas web contienen formularios HTML para la solicitud de información al usuario. El proceso de recolección de datos y respuesta de este formulario se debe realizar por medio de un programa. Esto lo hace un servlet que recoge las peticiones de un formulario y las procesa.

La declaración de un formulario HTML se hace por medio de la etiqueta `<form>`, donde es primordial que se especifique el método por el que se realizará el envío de los datos, los cuales pueden ser `get` o `post`.

A continuación, se presentan los aspectos más relevantes en lo relativo a formularios HTML con servlets:

Video 3. Formulario Servlet



[Enlace de reproducción del video](#)

Síntesis del video: Formulario Servlet

El video presenta un ejemplo práctico de cómo crear formularios HTML y procesarlos utilizando servlets en Java. Comienza con la creación de un proyecto en un entorno de desarrollo y la configuración de las opciones del servidor. Se explica la creación de un formulario HTML que solicita al usuario ingresar un nombre de usuario y una contraseña, destacando la importancia de la etiqueta `<form>` y la especificación del método (get o post) y la acción que apunta al servlet encargado de procesar los datos. También se muestra cómo crear un servlet que recibe y procesa

los datos enviados desde el formulario, sobrescribiendo el método doGet para manejar las peticiones y respuestas del usuario.

El video también cubre la construcción visual del formulario utilizando herramientas de arrastrar y soltar, especificando los nombres de los campos del formulario para su procesamiento en el servlet. Se detalla cómo el servlet recibe los datos del formulario a través de las variables request y response, almacenando los datos en variables y validándolos. Finalmente, se compara la información ingresada con valores predefinidos para verificar su validez y se envía una respuesta al usuario indicando si el registro fue exitoso o si hubo algún error. El video concluye mostrando la respuesta obtenida al enviar el formulario, validando así el funcionamiento del servlet tanto para datos correctos como incorrectos.

2.3. HTTP Session

Las sesiones se crean para hacer un seguimiento y memorizar las acciones de un usuario en un sitio web. Las sesiones en Java permiten guardar información entre las diversas peticiones HTTP, utilizando una clase que realiza el seguimiento de las sesiones; estos objetos también se almacenan del lado del servidor.

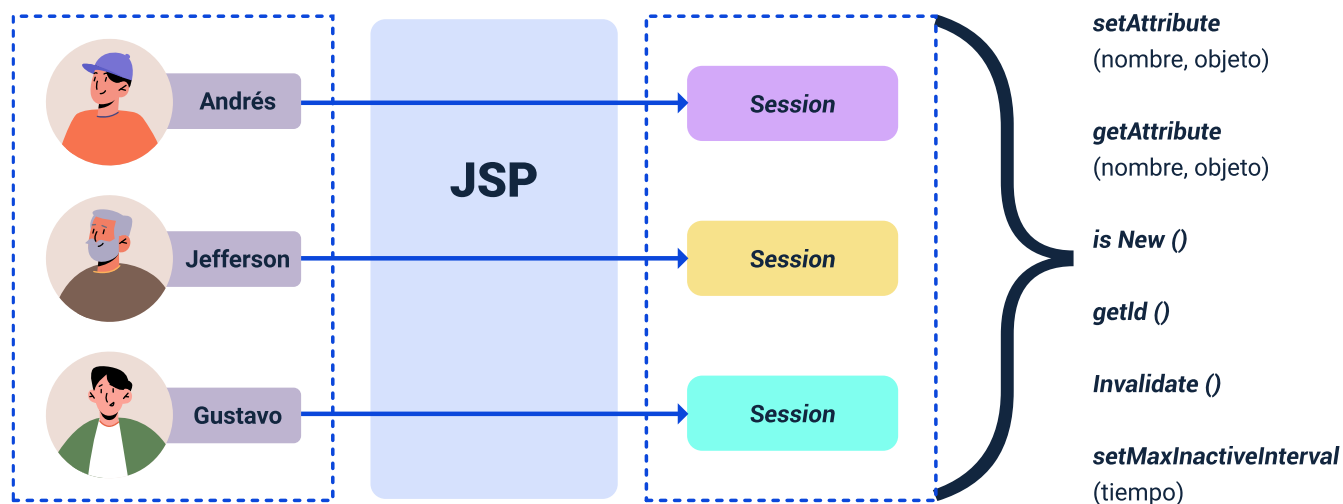
Para poder hacer uso de este objeto, es necesario obtener un objeto de la sesión para poder escribir en él, eliminarlo o leerlo, incluso después de que la sesión haya sido cerrada.

Los métodos más utilizados de HttpSession son:

- a) **getId()** Retorna el identificador que es asignado a la sesión. Se utiliza para la identificación de la sesión.
 - b) **isNew()** Retorna el valor booleano true en caso de que el cliente no tenga una sesión.
 - c) **setAttribute()** Establece un objeto en la sesión, empleando un nombre explícito.
 - d) **getAttribute()** Retorna el objeto que fue asignado a la sesión, identificado con el nombre que le fue asignado.
 - e) **setInactiveInterval()** Detalla el mayor tiempo que puede existir entre las peticiones continuas del cliente.
- El método utilizado para obtener la sesión de un usuario es `getSession()` este método regresa una interfaz de tipo `HttpSession`, Esta interfaz está dentro del paquete `javax. servlet.http` y es usada por el contenedor de páginas JSP que permite la creación de una sesión entre el cliente y el servidor.
`<%HttpSession session = request.getSession(); %>`

A continuación, se puede conocer el funcionamiento básico de las sesiones:

Figura 7. HTTP Session



- Analice detenidamente la gráfica e identifique cómo representa a tres usuarios que navegan por el mismo sitio web.
- Cada uno de estos tres usuarios accede al archivo JSP, el cual crea un objeto de sesión único para cada uno de ellos.
- La gráfica también presenta cómo estos objetos de sesión son guardados en la memoria del servidor. Estos permiten acceder a una serie de métodos que pertenecen a los objetos de tipo sesión.
- El método `setAttribute` tiene dos parámetros (nombre, objeto), donde el objeto es el que se almacenará en la sesión.
- Identifique cómo el método `getAttribute` devuelve el objeto que se ha almacenado en la sesión.
- Revise en la gráfica la representación de cómo el método `isNew` permite comprobar si la sesión es nueva o no.

2.4. Elementos de JSP

Las sesiones se crean para hacer un seguimiento y memorizar las acciones de un usuario en un sitio web. Las sesiones en Java permiten guardar información entre las diversas peticiones HTTP, utilizando una clase que realiza el seguimiento de las sesiones; estos objetos también se almacenan del lado del servidor. Para poder hacer uso de este objeto, es necesario obtener un objeto de la sesión para poder escribir en él, eliminarlo o leerlo, incluso después de que la sesión haya sido cerrada.

- **Códigos**

Se puede introducir código Java de varias clases como variables, métodos, sentencias, etc., y ser ejecutado en el contenedor JSP.

- **Directivas**

Brindan información general de la página y muestran lo que debe hacer el motor JSP con la página JSP. Para indicar una directiva se utiliza el símbolo arroba (@). Existen tres tipos: Page, Include y Taglib.

- **Acciones**

Facilitan al motor datos referentes a las acciones que debe realizar cuando se procesa la página (redirecciones). Hay 7 tipos: Param, Forward, Include, UseBean, getProperty, setProperty, Plugin.

- **Inserción de código Java**

Existen tres formas para insertar código Java en JSP: Expresiones `<%= expresión java %>`, Scriptlets `<% varias líneas de código java %>`, Declaraciones `<%! Declara variables o métodos %>`.

- **Expresiones**

Encerradas en la etiqueta `<%= expresión %>`, evalúan la expresión para convertir su resultado en un String, siendo este insertado en la salida. Solo tienen una línea de código. Ejemplos: `<%= new Date() %>`, `<%= 10+1 %>`, `<%= 10<4 %>`.

- **Scriptlets**

Inician con `<%` y terminan con `%>`. No tienen signo igual y pueden agregar varias líneas de Java, como bucles y condicionales. Ejemplo: `<% for(int i=0; i<10; i++) System.out.println("La cantidad es " + i); %>`.

- **Declaraciones**

Inician con `<%!` y terminan con `%>`. Se pueden escribir varias líneas de código y realizar declaraciones. Ejemplo: `<%! private int total; public int suma(int a, int b) { total = a + b; return total; } %>`.

Para afianzar sus conocimientos en elementos JSP, le invitamos a revisar el siguiente vídeo.

Video 4. Tipos de código JSP



[Enlace de reproducción del video](#)

Síntesis del video: Tipos de código JSP

El video Tipos Código JSP proporciona una guía sobre la inserción de código en páginas JSP. Comienza con la creación de un proyecto web en Eclipse y la configuración del servidor. Luego, detalla el trabajo con archivos JSP, que combinan HTML con Java. Se explican tres formas principales de inserción de código en JSP:

Expresiones: Utilizan `<%= %>`. Ejemplo: conversión de texto a mayúsculas.

Scriptlets: Bloques de código Java con `<% %>`. Ejemplo: bucles y sentencias de control.

Declaraciones: Definiciones de variables o métodos con `<%! %>`. Ejemplo: declaración y uso de métodos dentro de la página JSP.

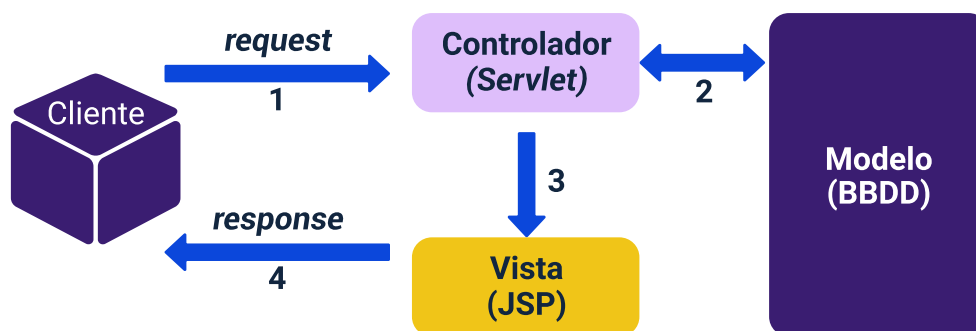
El video incluye ejemplos prácticos para ilustrar cada tipo de inserción de código, como operaciones matemáticas y manipulación de cadenas. Concluye con la combinación de estos métodos para la creación de aplicaciones web funcionales.

2.5. MVC

El MVC (Modelo-Vista-Controlador) es un patrón de diseño estándar que se utiliza para crear aplicaciones dinámicas con Java. Consiste en separar la parte lógica de los proyectos de la parte visual, integrando servlets y JSP. En este patrón, los servlets se encargan de la parte lógica y los JSP de la parte visual.

División de la aplicación realizada por el patrón:

Figura 8. Arquitectura MVC



- **El modelo**

Realiza el almacenamiento y la manipulación de los datos.

- **La vista**

Se encarga de la presentación con la que el usuario realizará la interacción, habitualmente en HTML.

- **El controlador**

Recoge los datos de entrada que envía el usuario, realiza la solicitud de acciones al modelo y modera la vista.

3. Fundamentos de React JS

React JS es una nueva tecnología importante para la construcción de software en su componente Front-end, que requiere tener en cuenta algunas consideraciones, como se pueden conocer a continuación:

- **¿Qué es?**

Es una nueva tecnología de reciente desarrollo, relacionada con una librería o biblioteca.

- **¿En qué se basa?**

Trabaja principalmente sobre componentes.

- **¿Cuál es su propósito?**

Inicialmente, fue pensada para solucionar el problema de interfaces que están actualizando información constantemente. La mayoría de las aplicaciones usadas en la actualidad actualizan información a cada momento mediante nuevos comentarios, notificaciones, publicaciones, imágenes, etc.

- **¿Quién la creó?**

Fue creada originalmente por Facebook, y hoy en día es una de las librerías más populares que existen para páginas web y front-end.

- **¿Cómo trabaja?**

Trabaja mediante una librería de JS. Una librería es una colección de procesos almacenados en objetos o módulos, es decir, códigos que aceleran el flujo de trabajo, pues consiste en reciclar algo que alguien ya ha creado antes.

¿Qué es React JS?

Es una biblioteca JavaScript de código abierto, desarrollada por la multinacional Facebook para crear interfaces de usuario. Es importante tener en cuenta que una biblioteca o librería es un conjunto de funcionalidades ya construidas y reutilizables siguiendo su sintaxis. Su principal objetivo, además de facilitar el desarrollo de aplicaciones de una sola página, es ser sencilla, declarativa y fácil de combinar.

- ¡Importante! Los datos deben ser atractivos, entendibles y accesibles para el usuario. Para esto, se deben tener en cuenta los objetivos de alistamiento y preparación de la información.

¿Qué es JSX?

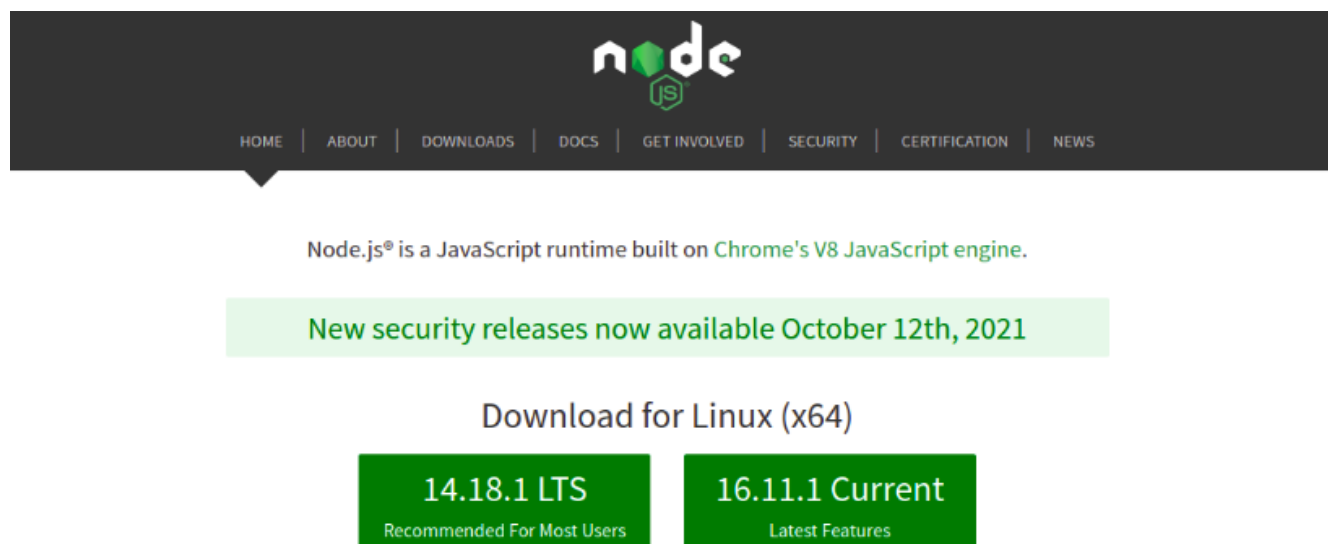
Su nombre viene de la composición de JavaScript (JS) + XML = (JSX) y corresponde a la sintaxis propuesta en React JS para escribir elementos de interfaz web compuestos por JavaScript + HTML, donde el HTML sigue las normas de código bien formado XML.

3.1. Instalando React JS

Las siguientes instrucciones presentan el proceso de instalación para sistemas operativos Linux versión Ubuntu 20.04.3 LTS y Windows. En ambos casos, primero se necesita instalar el servidor denominado Node y el manejador de paquetes npm.

- a) Ingrese a la página oficial de Node.js: <https://nodejs.org/en/>.

Figura 9. Sección de descargas en el sitio web de Node.js



b) Ingresa a la pestaña DOWNLOADS.

Figura 10. Página principal del sitio web de Node.js



Windows Installer
node-v14.18.1-x86.msi



macOS Installer
node-v14.18.1.pkg



Source Code
node-v14.18.1.tar.gz

Windows Installer (.msi)

Windows Binary (.zip)

macOS Installer (.pkg)

macOS Binary (.tar.gz)

Linux Binaries (x64)

Linux Binaries (ARM)

Source Code

32-bit	64-bit
32-bit	64-bit
64-bit	
64-bit	
64-bit	
ARMv7	ARMv8
node-v14.18.1.tar.gz	

Additional Platforms

Docker Image

Linux on Power LE Systems

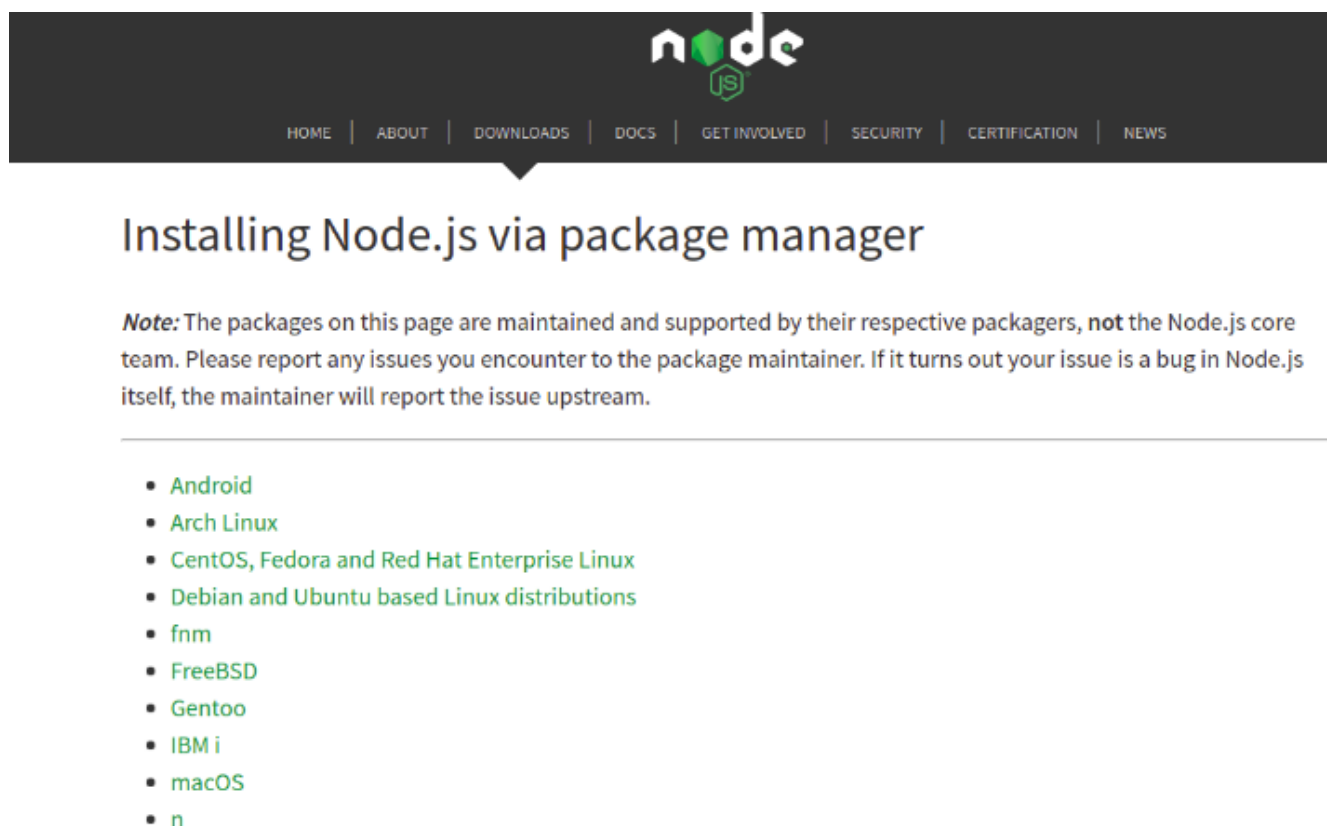
Linux on System z

AIX on Power Systems

Official Node.js Docker Image	
64-bit	
64-bit	
64-bit	

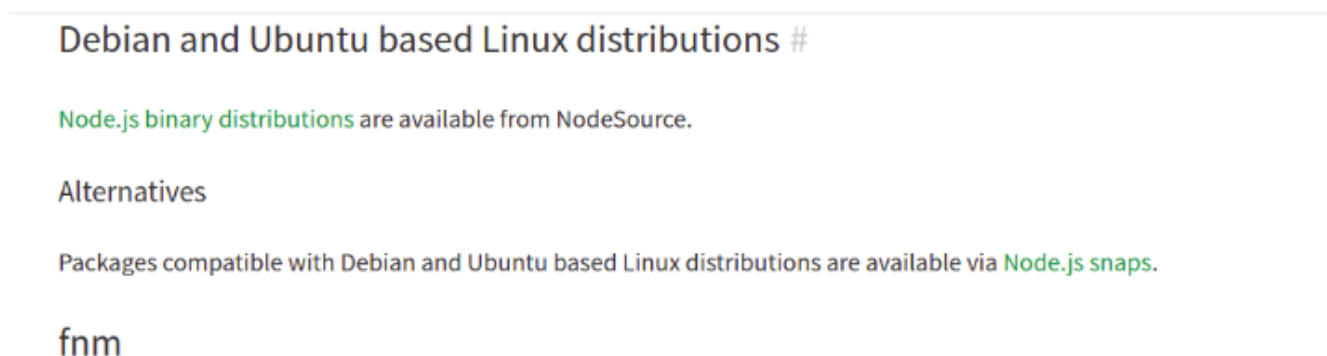
c) Ingrese a la opción Installing Node.js via package manager.

Figura 11. Información sobre la instalación de Node.js



d) Ingrese a la opción Debian and Ubuntu based Linux distributions.

Figura 12. Lista de sistemas operativos compatibles para la instalación de Node.js



e) Ingrese a la opción Node.js binary distributions.

Figura 13. Detalles específicos para distribuciones basadas en Debian y Ubuntu







PASSED

This repository contains documentation for using the [NodeSource Node.js](#) Binary Distributions via .rpm, .deb and Snap packages as well as their setup and support scripts.

If you are looking for NodeSource's low-impact Node.js performance monitoring platform, please [get started here](#).

Please file an issue if you are experiencing a problem or would like to discuss something related to the distributions.

Pull requests are encouraged if you have changes you believe would improve the setup process or increase compatibility across Linux distributions.

Table of Contents

- [Debian and Ubuntu based distributions](#) (deb)
 - [Installation instructions](#)
 - [Manual installation](#)
- [Enterprise Linux based distributions](#) (rpm)
 - [Installation instructions](#)
- [Tests](#)
- [FAQ](#)

f) Ingrese a la opción Installation instructions.

Figura 14. Página de NodeSource mostrando la compatibilidad de sus distribuciones binarias de Node.js

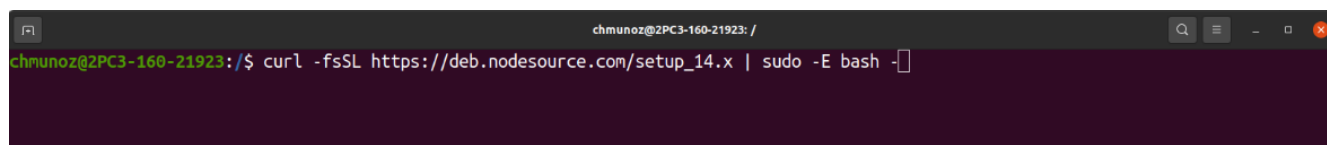
Node.js v14.x:

```
# Using Ubuntu
curl -fsSL https://deb.nodesource.com/setup_14.x | sudo -E bash -
sudo apt-get install -y nodejs

# Using Debian, as root
curl -fsSL https://deb.nodesource.com/setup_14.x | bash -
apt-get install -y nodejs
```

- g) Desde línea de comandos en Ubuntu, ingrese las siguientes instrucciones.
Si no tiene instalado curl, ingrese: `sudo snap install curl`.

Figura 15. Instrucciones de terminal para instalar Node.js en Ubuntu usando curl



```
chmunoz@2PC3-160-21923: /
chmunoz@2PC3-160-21923:/$ curl -fsSL https://deb.nodesource.com/setup_14.x | sudo -E bash -
```

- h) Vuelva a ejecutar: `curl -fsSL https://deb.nodesource.com/setup\_14.x`

Figura 16. Pantalla de terminal mostrando la ejecución del comando curl

```

chmunoz@2PC3-160-21923: ~/my-app
chmunoz@2PC3-160-21923:~/my-app$ npm run build
> my-app@0.1.0 build /home/chmunoz/my-app
> react-scripts build

Creating an optimized production build...
Compiled successfully.

File sizes after gzip:

 41.34 KB  build/static/js/2.8833cd6e.chunk.js
 1.62 KB  build/static/js/3.ff0311e9.chunk.js
 1.17 KB  build/static/js/runtime-main.1c420381.js
 591 B    build/static/js/main.6bc689e0.chunk.js
 556 B    build/static/css/main.a617e044.chunk.css

The project was built assuming it is hosted at /.
You can control this with the homepage field in your package.json.

The build folder is ready to be deployed.
You may serve it with a static server:

  npm install -g serve
  serve -s build

Find out more about deployment here:

  https://cra.link/deployment

chmunoz@2PC3-160-21923:~/my-app$

```

- i) Luego ejecute el comando: `sudo apt-get install -y nodejs`.

Figura 17. Pantalla de terminal mostrando la compilación exitosa de una aplicación Node.js

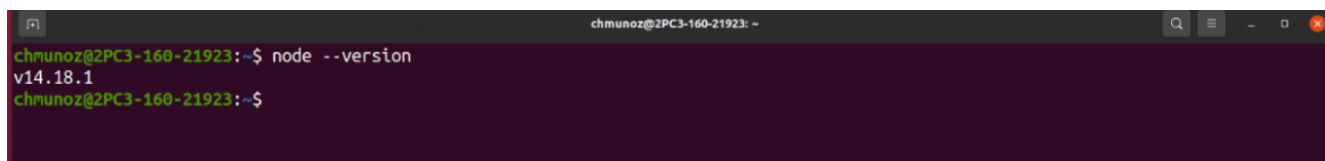
```

chmunoz@2PC3-160-21923: /
chmunoz@2PC3-160-21923:/$ sudo apt-get install -y nodejs
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes NUEVOS:
 nodejs
0 actualizados, 1 nuevos se instalarán, 0 para eliminar y 278 no actualizados.
Se necesita descargar 25,0 MB de archivos.
Se utilizarán 122 MB de espacio de disco adicional después de esta operación.
Des:1 https://deb.nodesource.com/node_14.x focal/main amd64 nodejs amd64 14.18.1-deb-1nodesource1 [25,0 MB]
Descargados 25,0 MB en 9s (2.687 kB/s)
Seleccionando el paquete nodejs previamente no seleccionado.
(Leyendo la base de datos ... 190600 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../nodejs_14.18.1-deb-1nodesource1_amd64.deb ...
Desempaquetando nodejs (14.18.1-deb-1nodesource1) ...
Configurando nodejs (14.18.1-deb-1nodesource1) ...
Procesando disparadores para man-db (2.9.1-1) ...
chmunoz@2PC3-160-21923:/$

```

- j) Finalmente, compruebe la versión instalada (v14.18.1) ejecutando el siguiente comando: `node --version`.

Figura 18. Terminal mostrando la instalación exitosa de Node.js en Ubuntu



```
chmunoz@2PC3-160-21923: ~$ node --version
v14.18.1
chmunoz@2PC3-160-21923: ~$
```

A continuación, siga los siguientes pasos de Instalación sobre sistema operativo Windows.

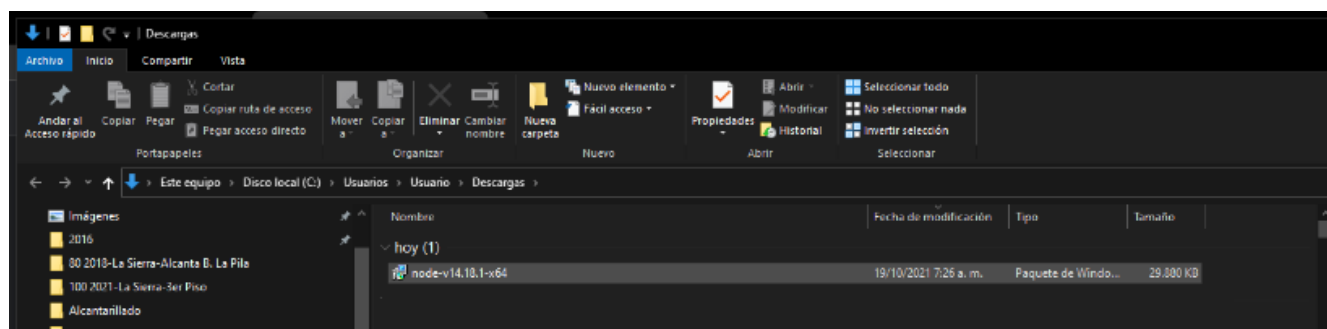
- **Paso 1.** Ingrese a Node >= 14.0.0 and npm >= 5.6 que corresponde a la página oficial de nodejs <https://nodejs.org/en/> y descargue la versión para windows de 64 bits.

Figura 19. Paso 1



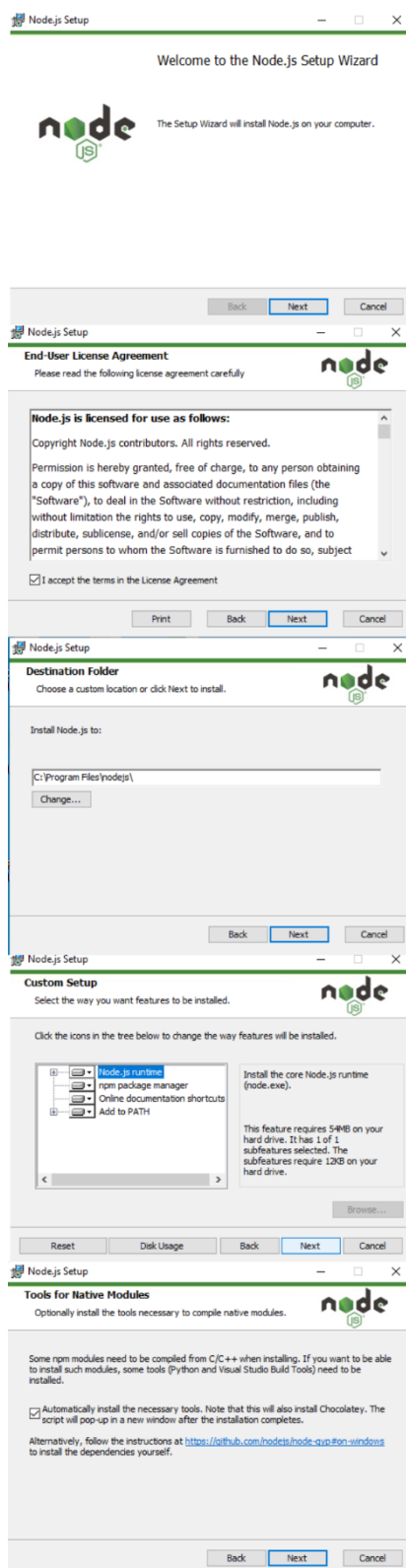
- **Paso 2.** Descargue en el sitio de su preferencia.

Figura 20. Paso 2



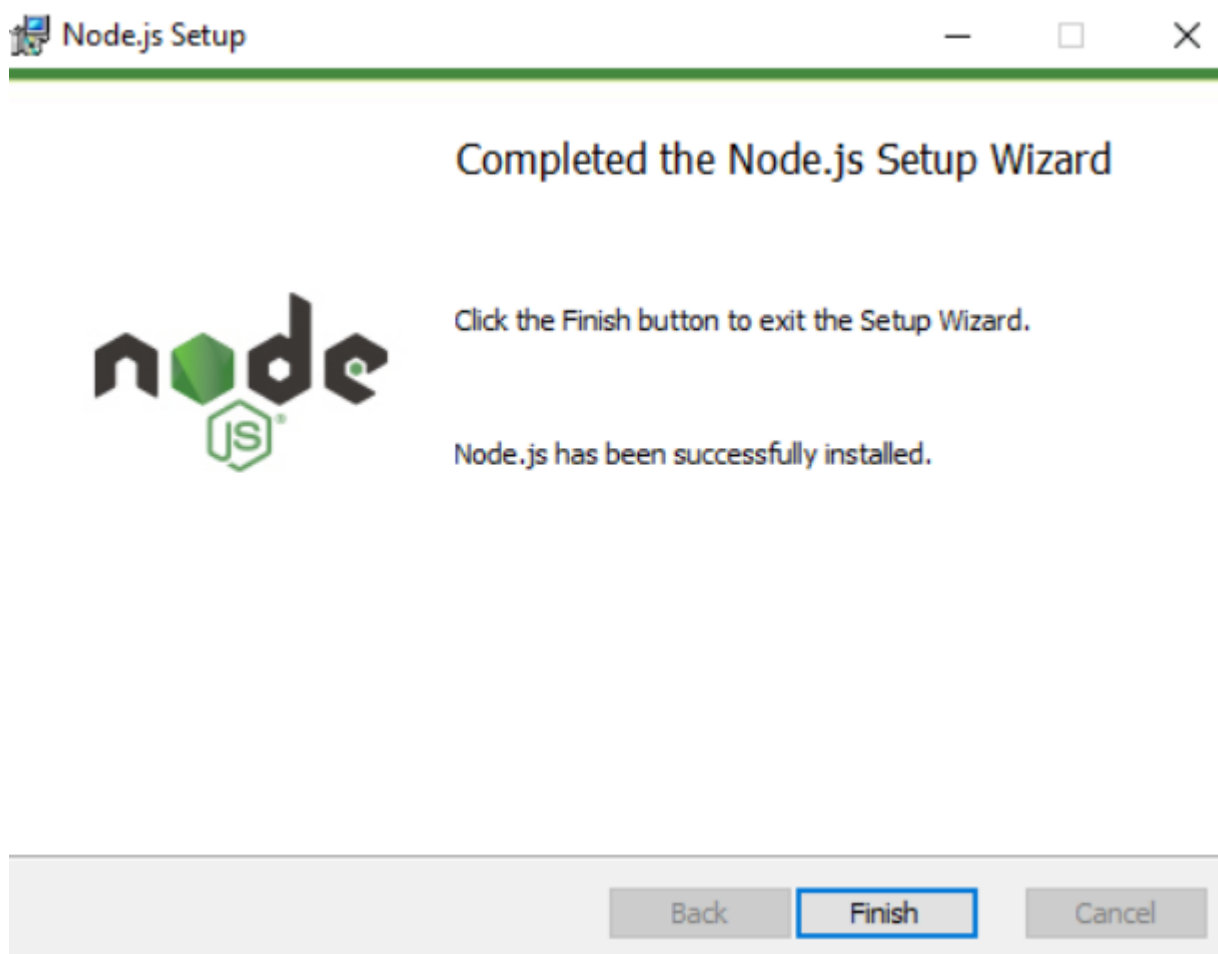
- **Paso 3.** Ejecute el archivo descargado como administrador de Windows y continúe el wizard haciendo clic en next en cada pantalla.

Figura 21. Paso 3



- **Paso 4.** Por último de clic en finish para terminar la instalación.

Figura 22. Paso 4



3.2. Crear un proyecto

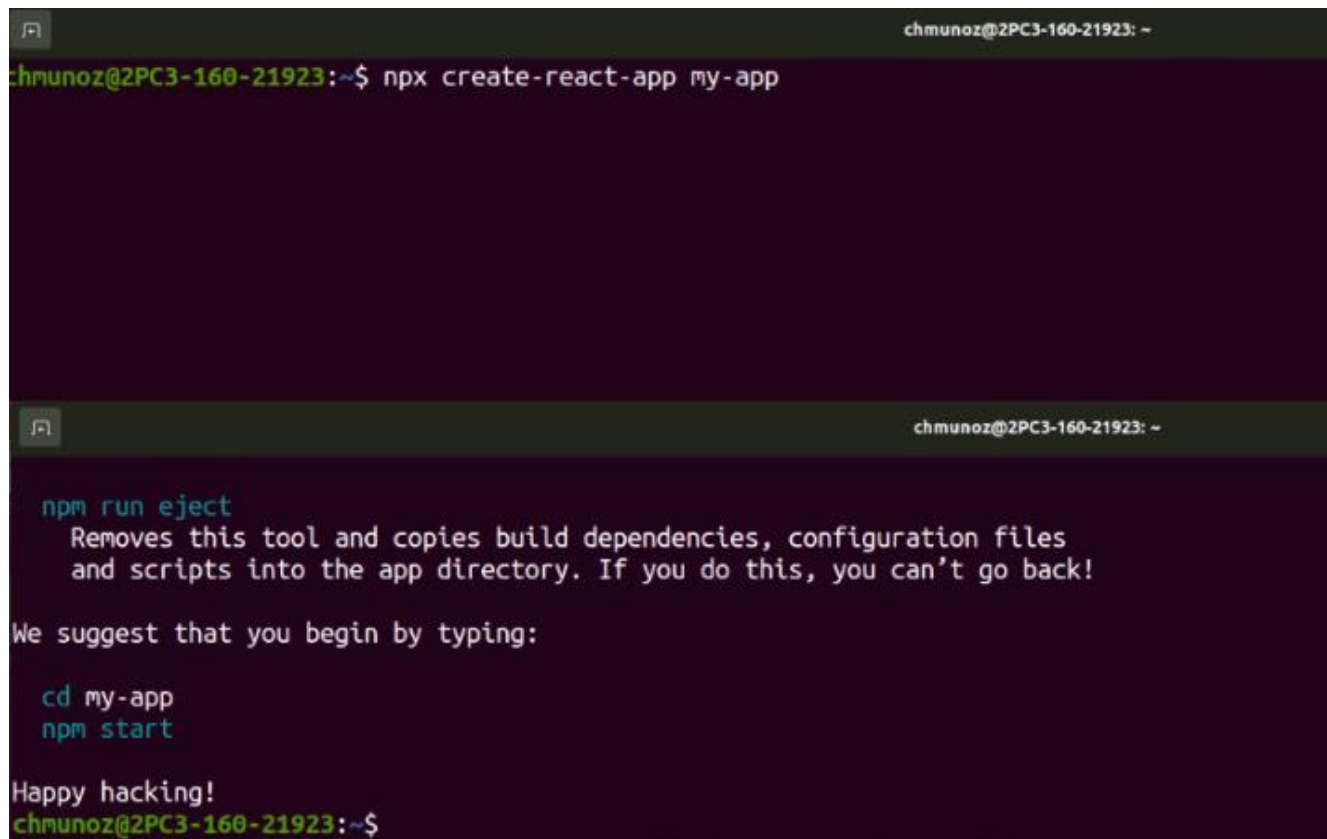
Una vez llevado a cabo el proceso de instalación del paso anterior de node y npm, se puede crear el proyecto React e ir ejecutando los siguientes comandos:

- `npx create-react-app my-app`
- `cd my-app`
- `npm start`

Iniciar la creación del proyecto, a través de la siguiente secuencia:

- **Paso 1.** Crear un comando Ubuntu: Desde la línea de comandos en Ubuntu, ejecute: `npx create-react-app my-app`.

Figura 23. Pantalla de terminal



```
chmunoz@2PC3-160-21923: ~$ npx create-react-app my-app

npm run eject
  Removes this tool and copies build dependencies, configuration files
  and scripts into the app directory. If you do this, you can't go back!

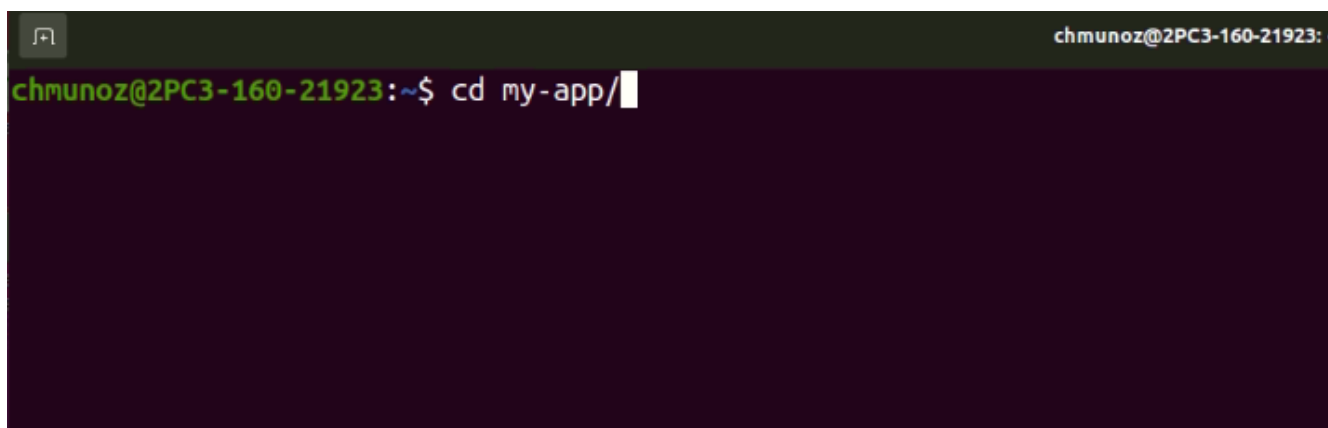
We suggest that you begin by typing:

  cd my-app
  npm start

Happy hacking!
chmunoz@2PC3-160-21923: ~$
```

- **Paso 2.** Ejecución de `npx create-react-app my-app`: Una vez el sistema termine la ejecución del paso anterior, ejecute: `cd my-app`.

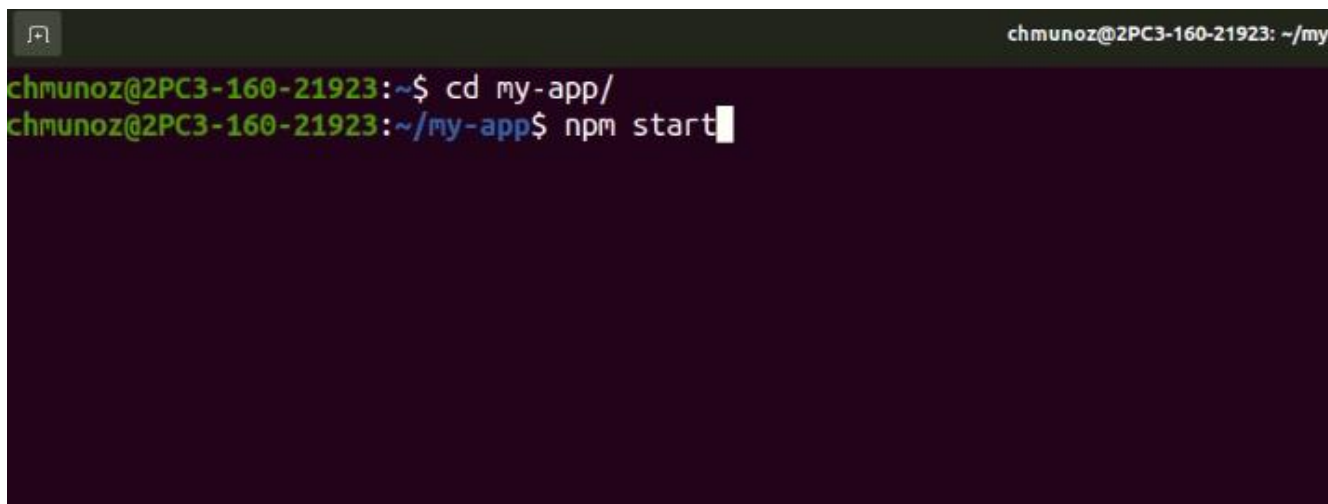
Figura 24. Pantalla de terminal después de navegar



```
chmunoz@2PC3-160-21923:~$ cd my-app/
```

- **Paso 3.** Ejecute npm start para lanzar el proyecto: Dentro de la carpeta, ejecute: npm start para lanzar el proyecto.

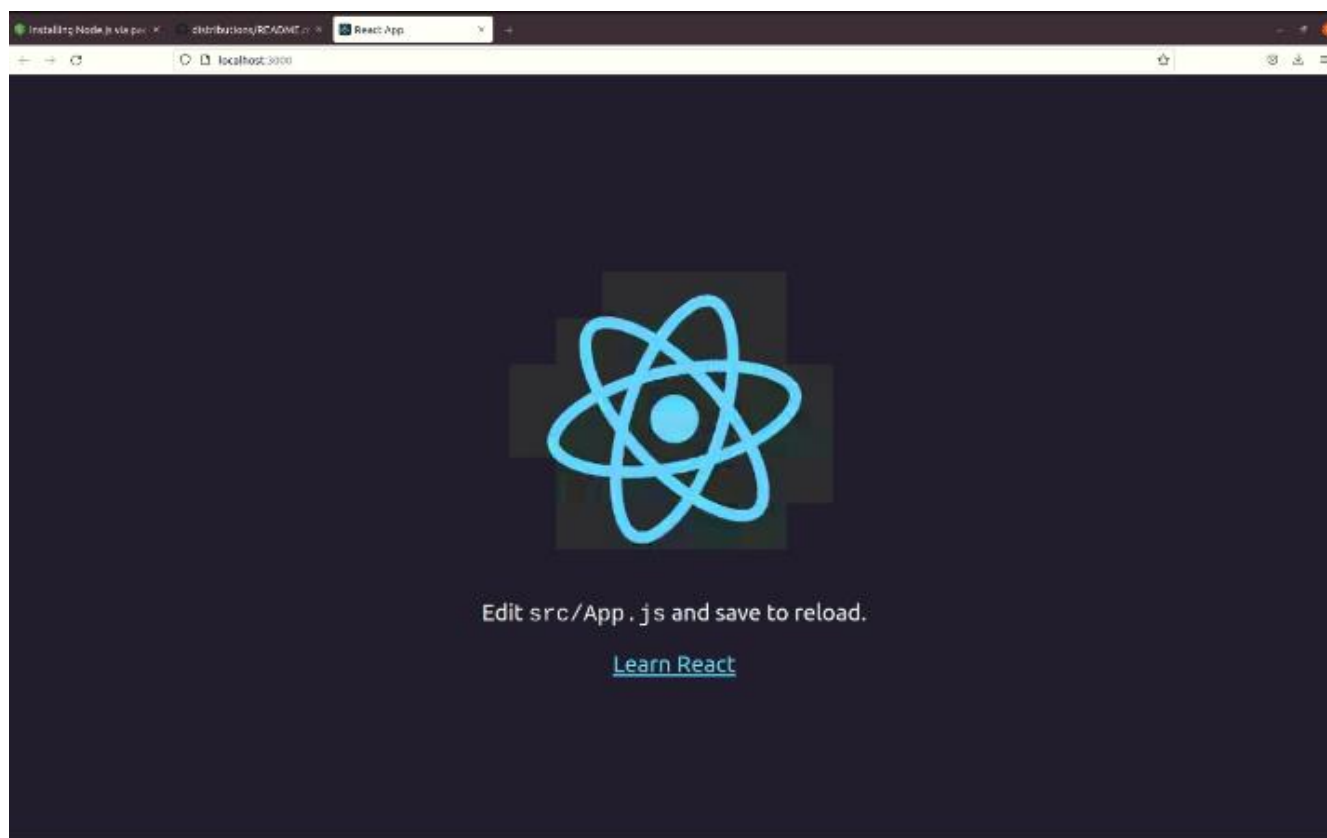
Figura 25. Pantalla de terminal mostrando el comando npm start ejecutándose



```
chmunoz@2PC3-160-21923:~$ cd my-app/
chmunoz@2PC3-160-21923:~/my-app$ npm start
```

- **Paso 4.** Proyecto React: El sistema lanza el primer proyecto React en el navegador como lo muestra la siguiente imagen.

Figura 26. Página inicial en el Navegador



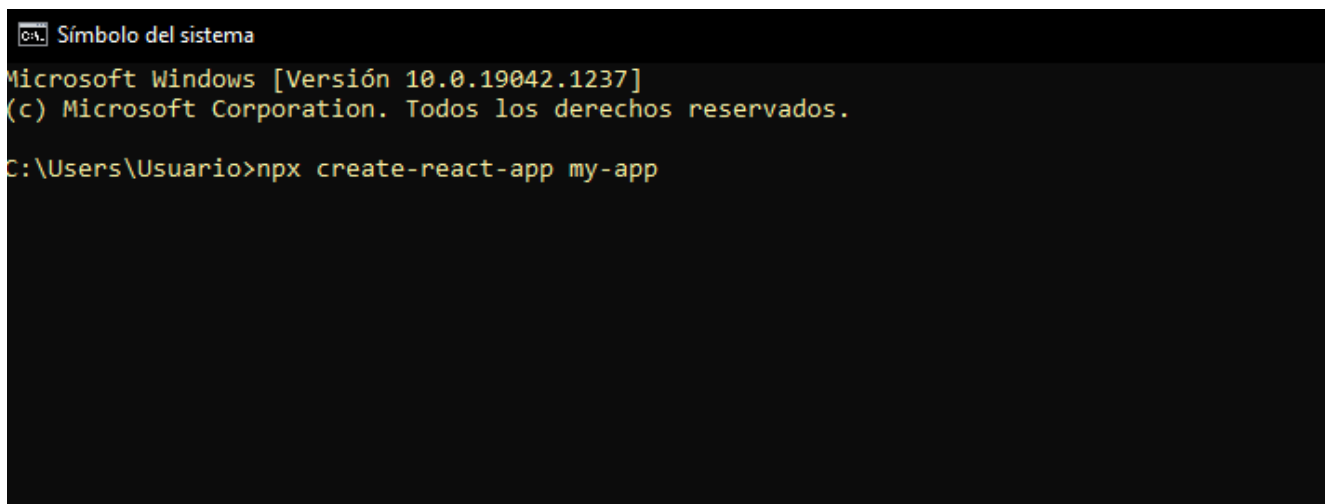
- **Paso 5.** Estructura del proyecto: Una vez finalizada la creación del proyecto, revise que la estructura del proyecto React esté conformada por el siguiente conjunto de carpetas y archivos.

Figura 27. Estructura del proyecto

```
my-app
├─ node_modules
│   └─ Conjunto de carpetas de los módulos de node
├─ package.json Archivo para gestionar las dependencias del proyecto
├─ package-lock.json
├─ public Carpeta en donde se encuentra recursos compartidos del proyecto
│   ├── favicon.ico
│   ├── index.html
│   ├── logo192.png
│   ├── logo512.png
│   ├── manifest.json
│   └─ robots.txt
├─ README.md Archivo de lectura y documentación del proyecto
└─ src Carpeta donde se encuentra el código fuente del proyecto
    ├── App.css
    ├── App.js
    ├── App.test.js
    ├── index.css
    ├── index.js
    ├── logo.svg
    ├── reportWebVitals.js
    └─ setupTests.js
```

- **Paso 6.** Crear un comando Windows: Desde la línea de comandos en Windows, ejecute: `npx create-react-app my-app`.

Figura 28. Pantalla de la consola de comandos de Windows



```

C:\> Símbolo del sistema

Microsoft Windows [Versión 10.0.19042.1237]
(c) Microsoft Corporation. Todos los derechos reservados.

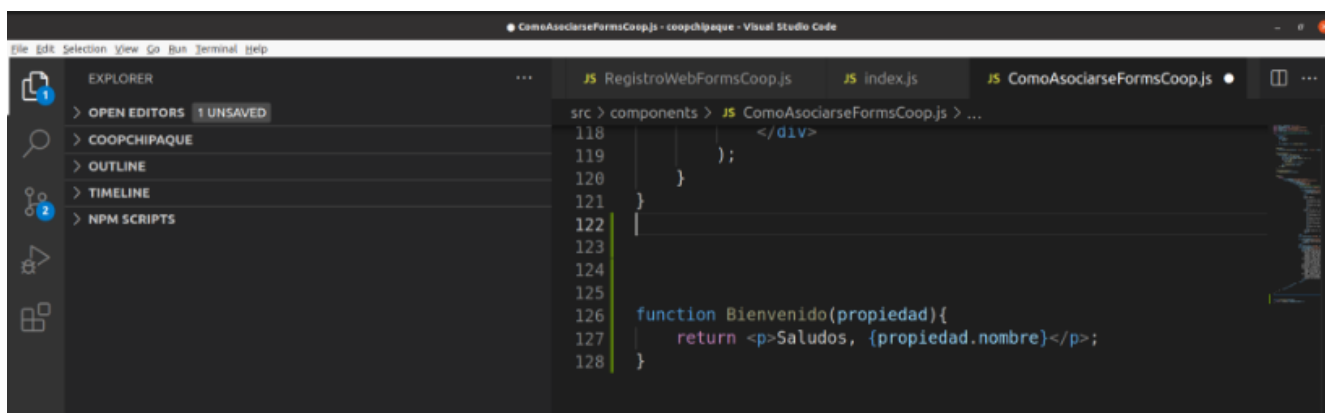
C:\Users\Usuario>npx create-react-app my-app
  
```

3.3. Propiedades y estados de los componentes

Un componente en React hace referencia a la posibilidad de definir de manera independiente una funcionalidad que puede ser reutilizable dentro del sistema. Esencialmente, estas funcionalidades se escriben como funciones JavaScript que reciben entradas denominadas propiedades y devuelven una forma de visualización.

Ejemplo de un componente denominado "Bienvenido" que recibe una propiedad y devuelve un párrafo de saludo.

Figura 29. Propiedades



```

src > components > JS ComoAsociarseFormsCoop.js > ...
118     </div>
119   );
120 }
121 }
122
123
124
125
126 function Bienvenido(propiedad){
127   return <p>Saludos, {propiedad.nombre}</p>;
128 }
  
```

Además, a un componente se le puede aplicar el concepto de estado en donde este, se define como la capacidad de especificar variables que pueden cambiar su valor en el tiempo, convirtiéndose en un comportamiento interno del componente, estos componentes se definen con el nombre de clase con la palabra reservada `class`.

```
class Clock extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = { date: new Date() };  
  
  }  
  
  render() {  
  
    return (  
  
      <div>  
  
        <h1>Hola, Mundo!</h1>  
  
        <h2>Es {this.state.date.toLocaleTimeString()}</h2>  
  
      </div>  
  
    );  
  
  }  
  
}
```

Eventos en React

Los eventos en React se manejan semejante a los del DOM se le pasa una función al controlador del evento.

```
function Bienvenido(propiedad) {  
    return <p>Saludos, {propiedad.nombre}</p>;  
}  
  
function activarCamaras(propiedad) {  
    return <p>Camara Uno, {propiedad.nombre}</p>;  
}
```

```
<button onClick={activarCamaras}>  
    Activar Camaras  
</button>
```

```
class Clock extends React.Component {  
    constructor(props) {  
        super(props);  
        this.state = { date: new Date() };  
    }
```

```
}  
  
render() {  
  return (  
    <div>  
      <h1>Hola, Mundo!</h1>  
      <h2>Es {this.state.date.toLocaleTimeString()}.</h2>  
    </div>  
  );  
}  
}
```

Listas y Keys

Las listas hacen referencia a un arreglo de datos, que puede ser un arreglo de objetos o de datos simples como caracteres, números, etc. Como toda lista en programación, React proporciona un método para extraer estos datos y mostrarlos en la página en la cual se está trabajando. A continuación, se presenta cómo se hace:

En React es necesario pasar arreglos de datos a listas para el manejo de algunos componentes lo cual se trabaja con la función `map` de la siguiente manera en donde se devuelve un elemento `<li>` por cada número.

Adicional al manejo de listas es muy importante tener en cuenta el identificador único de componentes para evitar errores en el DOM por lo tanto es necesario manejar el atributo key.

```
const numbers = [1, 2, 3, 4, 5];

const listItems = numbers.map((number) =>

  <li>{number}</li>

);

function NumberList(props) {

  const numbers = props.numbers;

  const listItems = numbers.map((number) =>

    <li key={number.toString()}>

      {number}

    </li>

  );

  return (

    <ul>{listItems}</ul>

  );

}
```

Formularios

En React el envío de formularios se comporta como en HTML, solo que aquí, el ingreso y cambio de valores en las entradas se deben manejar con componentes controlados por medio de estados.

```
class Clock extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = { date: new Date() };  
  
  }  
  
  render() {  
  
    return (  
  
      <div>  
  
        <h1>Hola, Mundo!</h1>  
  
        <h2>Es {this.state.date.toLocaleTimeString()}.</h2>  
  
      </div>  
  
    );  
  
  }  
  
}
```

Concepto de clases en React

Las clases son otra forma de definir un componente en React. Se caracterizan por tener propiedades y, a diferencia de las funciones, pueden definir estados propios sin utilizar el concepto de Hook. Dentro de los componentes de clase, existe una función constructora que se utiliza para definir el estado inicial del componente. Es muy importante llamar a la función `super()` dentro del constructor para que los parámetros funcionen adecuadamente, como puede verse en la siguiente imagen:

```
class Counter extends React.Component {  
  
  constructor(props) {  
  
    super(props);  
  
    this.state = { foo: 123, bar: 456 };  
  
  }  
  
  render() {  
  
    return (  
  
      <div>foo: {this.state.foo} bar: {this.state.bar}</div>  
  
    );  
  
  }  
  
}
```

Explorar el siguiente recurso y conocer los aspectos más relevantes relacionados con el concepto de clases en React:

- Los componentes permiten separar la interfaz de usuario en piezas independientes, reutilizables y pensar en cada pieza de forma aislada.
- React permite definir componentes como clases o funciones. Los componentes definidos como clases actualmente proporcionan una serie de características extra.
- La función `super()` dentro del constructor es un componente de React válido porque acepta un solo argumento de objeto "props" con datos y devuelve un elemento de React. Llamamos a dichos componentes "funcionales" porque literalmente son funciones JavaScript.
- Nombres de componentes: Comience siempre los nombres de componentes con una letra mayúscula. React trata los componentes que empiezan con letras minúsculas como etiquetas del DOM. Por ejemplo, `<div/>` representa una etiqueta div HTML pero `<Welcome/>` representa un componente y requiere que `Welcome` esté definido.

Funciones Hooks

Las funciones Hooks permiten a los componentes definidos por medio de funciones manejar variables de estado locales, similar a cómo lo hacen las clases. Los Hooks son funciones JavaScript y están sujetas a dos reglas:

- Solo llamar Hooks en el nivel superior. No se deben llamar Hooks dentro de loops, condiciones o funciones anidadas.
- Solo llamar Hooks desde componentes de función de React. No se deben llamar Hooks desde las funciones regulares de JavaScript.

Ejemplo de una función que utiliza el Hook useState:

```
import React, { useState } from 'react';

function Example() {

  // Declara una nueva variable de estado, que llamaremos "count"

  const [count, setCount] = useState(0);

  return (

    <div>

      <p>You clicked {count} times</p>

      <button onClick={() => setCount(count + 1)}>

        Click me

      </button>

    </div>

  );

}
```

Ciclos de vida de los componentes de React

Hace referencia a los métodos que se pueden utilizar por defecto (es decir, que forman parte de la librería de React). Cuando se ejecuta o se renderiza un componente en la línea de tiempo de ejecución del programa, estos métodos son:

Tabla 2. Métodos de ciclo de vida de React

Métodos de ciclo de vida de React	Descripción
Nombre del Método	Utilidad
componentWillMount()	Se ejecuta justo antes del primer pintado del componente.
componentWillReceiveProps(nextProps)	Se invoca cuando las propiedades se van a actualizar.
shouldComponentUpdate(nextProps, nextState)	Se invocará cuando se producen cambios de propiedades o cambios de estado.
componentWillUpdate(nextProps, nextState)	Este se invocará justo antes de que el componente vaya a actualizar su vista.
componentDidUpdate(prevProps, prevState)	Este se invocará justo después de que el componente vaya a actualizar su vista.
componentWillUnmount()	Se ejecuta en el momento que el componente se va a retirar del DOM.

Se puede revisar en el siguiente código cómo quedará el archivo:

```
import React from 'react';
```

```
export default class CicloVida extends React.Component {
```

```
constructor(props) {  
  console.log('Ejecutando constructor', ...args);  
  super(props);  
  this.state = {  
    estado: 'Inicializado en el constructor'  
  };  
}
```

```
componentDidMount() {  
  console.log('Ejecuta componentDidMount');  
}
```

```
componentWillUnmount() {  
  console.log('Ejecuta componentWillUnmount');  
}
```

```
componentDidUpdate(prevProps, prevState) {  
  console.log('Ejecuta componentDidUpdate. Anteriores propiedades y  
estado:', prevProps, prevState);
```

```
}
```

```
shouldComponentUpdate(nextProps, nextState) {
```

```
    console.log('Ejecuta shouldComponentUpdate. Próximas propiedades y  
estado:', nextProps, nextState);
```

```
    return true;
```

```
}
```

```
componentWillReceiveProps(nextProps) {
```

```
    console.log('Ejecuta componentWillReceiveProps con las propiedades  
futuras:', nextProps);
```

```
}
```

```
componentWillUpdate(nextProps, nextState) {
```

```
    console.log('Ejecuta componentWillUpdate. Próximas propiedades y  
estado:', nextProps, nextState);
```

```
}
```

```
componentDidCatch(error, info) {
```

```
    console.log('Ejecuta componentDidCatch:', error, info);
```

```
}

render() {

  console.log('Ejecuta render');

  return (

    <div>

      <h1>Ciclo de vida del componente</h1>

      <p>Estado: {this.state.estado}</p>

      <p>Propiedad: {this.props.propiedad}</p>

    </div>

  );

}
```

¡Atención! En React.js los componentes que no sean puros (todos los que se crean mediante clases o `React.createClass`) poseen algo conocido como el ciclo de vida. Este ciclo de vida, son una serie de funciones que se ejecutan en distintos momentos de la vida del componente y nos permiten realizar distintas acciones en estos momentos.

Ejecutar la aplicación

Para ejecutar una aplicación React JS, se utiliza el comando `npm start`. Para generar la versión de producción, se debe ejecutar el siguiente comando: `npm run build`.

```
> my-app@0.1.0 build /home/chmunoz/my-app
```

```
> react-scripts build
```

```
Creating an optimized production build...
```

```
Compiled successfully.
```

```
File sizes after gzip:
```

```
41.34 KB build/static/js/2.8833cde.chunk.js
```

```
1.62 KB build/static/js/3.ff0311e9.chunk.js
```

```
1.38 KB build/static/js/runtime~main.1c420381.js
```

```
1.31 KB build/static/js/main.68b9e600.chunk.js
```

```
506 B build/static/css/main.cfa70044.chunk.css
```

```
The project was built assuming it is hosted at /.
```

```
You can control this with the homepage field in your package.json.
```

The build folder is ready to be deployed.

You may serve it with a static server:

```
npm install -g serve
```

```
serve -s build
```

Find out more about deployment here:

<https://cra.link/deployment>

Este comando genera la carpeta build con la versión compilada y minificada del proyecto React SPA (Single Page Application). A continuación, se describirá cómo debe quedar la organización y el registro de este proceso.

build

└─ asset-manifest.json

└─ favicon.ico

└─ index.html

└─ logo192.png

└─ logo512.png

- |— manifest.json
- |— robots.txt
- └─ static
 - |— css
 - | |— main.a617e044.chunk.css
 - | |— main.a617e044.chunk.css.map
 - |— js
 - | |— 2.8833cd6e.chunk.js
 - | |— 2.8833cd6e.chunk.js.LICENSE.txt
 - | |— 2.8833cd6e.chunk.js.map
 - | |— 3.ff0311e9.chunk.js
 - | |— 3.ff0311e9.chunk.js.map
 - | |— main.6bc689e0.chunk.js
 - | |— main.6bc689e0.chunk.js.map
 - | |— runtime-main.1c420381.js
 - | |└─ runtime-main.1c420381.js.map
- └─ media
 - └─ logo.6ce24c58.svg

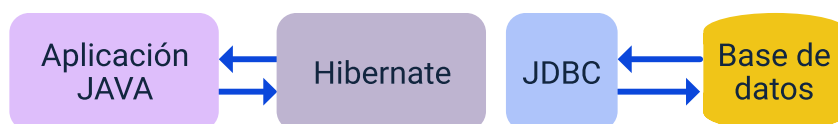
4. Framework Hibernate

Hibernate es software libre con licencia GNU LGPL que presta servicios de ORM para Java. Es un plugin que puede ser integrado al IDE de desarrollo en Java y facilita el proceso de mapeo de tablas en una base de datos relacional a objetos Java mediante el uso de archivos declarativos en formato XML o anotaciones.

“Hibernate utiliza JDBC para todos los procesos de comunicación con la base de datos, funcionando como una capa adicional de abstracción de las funcionalidades del JDBC. En una aplicación Java, podremos delegar la mayoría de las operaciones de bases de datos al framework de Hibernate, pero en segundo plano todo estará pasando por la API de JDBC” (Martinez, 2015).

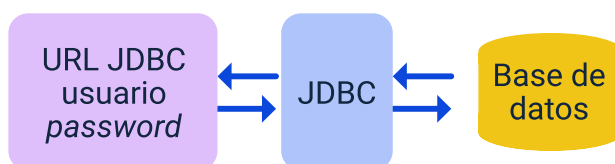
Así que, cuando se configura Hibernate para conectarse a una base de datos, lo que realmente se está haciendo es configurar Hibernate para el uso del JDBC, y es este quien realiza las operaciones. A continuación, se presenta el esquema general del funcionamiento:

Figura 30. Hibernate



Teniendo en cuenta que Hibernate debe usar el driver JDBC para la ejecución de las instrucciones en la base de datos, se debe construir un archivo de configuración en formato XML donde se especifique la URL del JDBC y las credenciales de acceso a la base de datos. En la siguiente figura se presenta el esquema general de funcionamiento en el proceso de configuración:

Figura 31. Hibernate archivo de configuración



Este archivo de configuración define un conjunto de propiedades que serán utilizadas para crear las sesiones mediante las cuales se inician un conjunto de transacciones u operaciones sobre la base de datos. Entre las propiedades más importantes se encuentran las siguientes:

Tabla 3. Propiedades Hibernate

Métodos de ciclo de vida de React	Descripción
connection.driver.class	Se ejecuta justo antes del primer pintado del componente.
connection.url	Se invoca cuando las propiedades se van a actualizar.
connection.username	Se invocará cuando se producen cambios de propiedades o cambios de estado.
connection.password	Este se invocará justo antes de que el componente vaya a actualizar su vista.
connection.pool_size	Este se invocará justo después de que el componente vaya a actualizar su vista.
Dialect	Se ejecuta en el momento que el componente se va a retirar del DOM.
show_sql	Se ejecuta en el momento que el componente se va a retirar del DOM.
current_session_context_class	Especifica el contexto de la sesión actual, por defecto se puede asignar thread.

Es importante tener en cuenta que este archivo de configuración de Hibernate debe estar ubicado en la raíz de la carpeta src del proyecto, de forma que pueda ser utilizado por Hibernate en cualquier contexto de la aplicación. A continuación, se muestra cómo debe verse un archivo de configuración de Hibernate completo, donde se establece una conexión con una base de datos MySQL en un servidor que se está ejecutando en la máquina local (localhost) y puerto 3306, con el usuario “root” y contraseña 12345:

```
<!DOCTYPE hibernate-configuration PUBLIC
```

```
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
```

```
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
```

```
<hibernate-configuration>
```

```
<session-factory>
```

```
<property
```

```
name="connection.driver_class">com.mysql.cj.jdbc.Driver</property>
```

```
<property
```

```
name="connection.url">jdbc:mysql://localhost:3306/prueba?useSSL=false&serverTime  
zone=UTC</property>
```

```
<property name="connection.username">root</property>
```

```
<property name="connection.password">12345</property>
```

```
<!-- JDBC connection pool settings ... using built-in test pool -->
```

```
<property name="connection.pool_size">1</property>

<!-- Select our SQL dialect -->

<property name="dialect">org.hibernate.dialect.MySQLDialect</property>

</session-factory>

</hibernate-configuration>

<!-- Echo the SQL to stdout -->

<property name="show_sql">true</property>

<!-- Set the current session context -->

<property name="current_session_context_class">thread</property>

</session-factory>

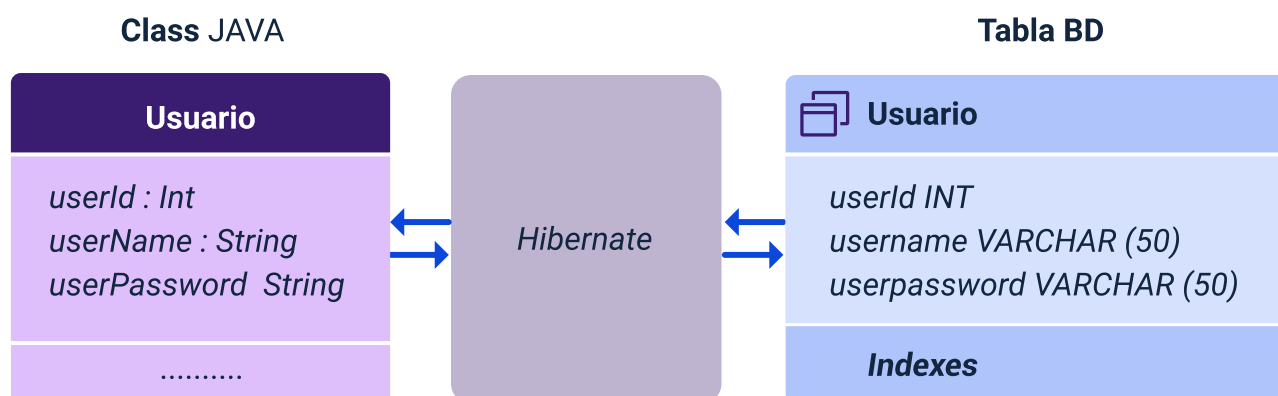
</hibernate-configuration>
```

Se invita a revisar en los anexos el archivo de configuración mostrado anteriormente para que pueda ser descargado y utilizado en la construcción de su proyecto, teniendo en cuenta que debe editar las propiedades de acuerdo con sus necesidades.

Luego de esto, se requiere definir todas las “Entity Classes” que serán utilizadas por Hibernate para realizar el proceso de mapeo hacia una base de datos relacional y viceversa. Una “Entity Class” básicamente es una clase Java tradicional que tiene las mismas características definidas en una tabla de la base de datos, es decir, es una imagen de la tabla, pero desde la perspectiva de la programación orientada a objetos.

Adicionalmente, tiene unas anotaciones especiales que le permiten a Hibernate realizar el trabajo adecuado.

Figura 32. Entity Classes



Las anotaciones pueden ser utilizadas al importar la librería de persistencia de JAVA `javax.persistence.*`, entre las más utilizadas encontramos:

Tabla 4. Anotaciones Java

Anotación	Significado
@Entity	Especifica que la clase definida en una Entity Class.
@Table(name = " ")	Permite especificar que la clase será mapeada con una tabla en la base de datos especificada en la propiedad name.
@Column(name = " ")	Permite especificar que el atributo por ser definido será mapeado con una columna especificada por la propiedad name. Cuando el nombre del atributo coincide con el nombre de la columna en la tabla se puede omitir la especificación de la propiedad name y solo dejar la anotación @Column.
@Id	Permite especificar que el atributo a definir corresponde con el identificador primario a ser mapeado en la tabla.

A continuación, se presenta un corto video donde se agrega el archivo de configuración de Hibernate al proyecto y se crea la clase Entity para la gestión de una tabla en una base de datos MySQL.

Video 5. Creación Clase Entity



[Enlace de reproducción del video](#)

Síntesis del video: Creación Clase Entity

En el video "Creación Clase Entity", se explica cómo configurar un proyecto para incluir un archivo de configuración de Hibernate y definir las clases de entidad necesarias para manipular una base de datos MySQL. Primero, se reutiliza un archivo de configuración previamente definido, el cual se coloca en la raíz del directorio de recursos del proyecto. Este archivo define propiedades esenciales para la conexión,

como el Driver Class y la URL de conexión. Luego, se crea una nueva clase de entidad en Java que refleja la estructura de la tabla usuario en la base de datos. La clase se organiza en un paquete llamado `com.ejemplo.entidad` y contiene los atributos `userID`, `username`, y `userPassword`. También se definen un constructor por defecto y un constructor parametrizado, además de los métodos `set` y `get` para cada atributo.

Posteriormente, se añaden anotaciones a la clase de entidad para especificar que es una entidad (`@Entity`) y se mapea a la tabla usuario utilizando la anotación `@Table`. Cada atributo se mapea a su columna correspondiente en la base de datos mediante las anotaciones `@Id` y `@Column`. Una vez definidas las clases de entidad, se pueden construir las operaciones de inserción, eliminación y actualización en la base de datos utilizando la configuración establecida. El video proporciona una guía detallada sobre cómo integrar Hibernate en un proyecto Java y definir clases de entidad para interactuar con una base de datos MySQL, permitiendo manipular los datos de manera eficiente y estructurada.

4.1. Operaciones básicas

Una vez se tienen registradas todas las dependencias, los archivos de configuración de Hibernate y las respectivas clases de entidad, se pueden utilizar las facilidades ofrecidas por Hibernate para la manipulación de la base de datos de forma transparente por medio de los procesos de mapeo que ofrece este framework.

Hibernate utiliza principalmente dos clases que debemos conocer y utilizar, las cuales se describen a continuación:

- **SessionFactory**

Esta clase lee el archivo de configuración de Hibernate y crea los objetos de sesión. Por ser una clase que genera objetos pesados por los procesos de los que es responsable solo debo crearlo una vez por toda la aplicación.

- **Session**

Esta clase envuelve todo lo correspondiente a una conexión JDBC y es el tipo de objeto que será usado para guardar o recibir objetos desde y hacia la base de datos. Es instanciado a partir de una clase SessionFactory. Los objetos creados de este tipo son más livianos por lo que pueden ser creados y desechados tantas veces como se necesite dentro de la aplicación.

En el siguiente fragmento de código se detalla cómo es el proceso de definición de los objetos SessionFactory y Session para poder realizar la manipulación de la base de datos:

```
// creación del session factory

SessionFactory factory = new Configuration()

    .configure("hibernate.cfg.xml")

    .addAnnotatedClass(Usuario.class)

    .buildSessionFactory();

// creación de la session a partir del Session factory

Session session = factory.getCurrentSession();
```

```
try {  
  
    // Ahora se puede usar el objeto session para la manipulación de la BD  
  
} finally {  
  
    // Se cierra el objeto session factory una vez se deja de utilizar  
  
    factory.close();  
  
}
```

Note que en el código anterior, para la creación del objeto `SessionFactory`, se indica el nombre del archivo de configuración de Hibernate en el método `configure()`. Si este parámetro no se incluyera, Hibernate buscará en el directorio raíz del proyecto un archivo que contenga el nombre por defecto de este tipo de archivos de configuración, el cual es `hibernate.cfg.xml`.

También note que en el método `addAnnotatedClass()` se indica la clase de entidad a ser utilizada.

Método Save

El método `save()` del objeto `Session` permite realizar el proceso de inserción de un nuevo registro en la base de datos enviando como parámetro un objeto de tipo entidad con sus valores. La forma correcta de realizar un proceso de inserción de un nuevo registro en una base de datos con un objeto de tipo sesión incluye los siguientes pasos:

- a) Iniciar una transacción por medio del método `beginTransaction()`.
- b) Ejecutar la lógica necesaria para crear un objeto de la clase de entidad con sus correspondientes valores.
- c) Invocar el método `save()` de la sesión enviando como parámetro el objeto de la clase entidad.
- d) Confirmar la transacción realizada por medio del método `commit()`.

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.jonathan.hibernate.demo.entity.Usuario;

public class CrearUsuarioDemo {

    public static void main(String[] args) {

        // Crear SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();
```

```
// Crear sesión

Session session = factory.getCurrentSession();

try {

    System.out.println("Creando un usuario...");

    // Crear un objeto Usuario

    Usuario usuario1 = new Usuario("Francisco", "FFFFF");

    // Iniciar una transacción

    session.beginTransaction();

    // Guardar el objeto Usuario

    System.out.println("Guardando el nuevo usuario...");

    session.save(usuario1);

    // Confirmar la transacción

    session.getTransaction().commit();
```

```
        System.out.println("Proceso finalizado.");  
  
    } finally {  
  
        factory.close();  
  
    }  
  
}  
  
}
```

Método Get

El método `get()` del objeto `session` permite obtener un registro de la base de datos por medio de una consulta por su llave primaria. Este método recibe como parámetro la clase de entidad que será utilizada para hacer el mapeo del registro en la respuesta y un segundo parámetro que corresponde al valor a ser buscado en la llave primaria de la tabla. Este método ofrece como resultado un objeto de la clase de entidad especificada en la invocación del método.

La forma correcta de realizar un proceso de consulta de un registro particular por medio de su llave primaria y un objeto de tipo sesión incluye los siguientes pasos:

- a) Iniciar una transacción por medio del método `beginTransaction()`.
- b) Invocar el método `get()` de la sesión, enviando como parámetro la clase de tipo entidad para hacer el mapeo y el valor de la llave primaria a buscar. La invocación de este método requiere de un objeto compatible que reciba el resultado.

- c) Confirmar la transacción realizada por medio del método
`getTransaction().commit()`.

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.jonathan.hibernate.demo.entity.Usuario;


public class CrearLeerDemo {


    public static void main(String[] args) {

        // Crear SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();


        // Crear sesión

        Session session = factory.getCurrentSession();
```

```
try {  
  
    // Iniciar una transacción  
  
    session.beginTransaction();  
  
  
    // Consulta de un usuario cuyo pk tiene el valor de 16  
  
    System.out.println("Consulta de un usuario");  
  
    Usuario resultado = session.get(Usuario.class, 16);  
  
    System.out.println("Resultado = " + resultado);  
  
  
    // Confirmar la transacción  
  
    session.getTransaction().commit();  
  
    System.out.println("Proceso finalizado.");  
  
} finally {  
  
    factory.close();  
  
}  
  
}
```

Método CreateQuery

Hibernate permite el proceso de creación de consultas usando HQL, el cual es una versión especial de Hibernate para establecer consultas SQL orientadas a objetos y que comprende nociones propias del paradigma orientado a objetos, como la herencia, el polimorfismo y las asociaciones. Es importante tener en cuenta que, aunque las palabras reservadas y sentencias de SQL no son sensibles a mayúsculas, los nombres de las clases y sus atributos usados en HQL sí lo son.

Para el caso en el que se requiera realizar consultas más avanzadas, se puede utilizar la potencia de HQL para establecer las características de los registros a ser consultados. Como este tipo de consulta puede producir más de un registro proveniente de la base de datos, es necesario usar el método `getResultList()` para mapear la respuesta a una lista de objetos Java. En la siguiente tabla se presentan algunas de las cláusulas HQL más comúnmente utilizadas. Se debe tener en cuenta que en HQL se hace referencia a las clases de entidad Java con sus atributos, los cuales son sensibles a mayúsculas. Estas sentencias son mapeadas a su estructura equivalente en la base de datos usando los nombres de tablas y atributos, los cuales ya no son sensibles a mayúsculas.

Tabla 5. Sentencias y equivalentes

Sentencia HQL	Equivalente en SQL
from Usuario	SELECT * FROM usuario
from Usuario u where u.userName = 'Raul'	SELECT * FROM usuario U WHERE u.username = 'Raul'
from DomesticCat cat	SELECT * FROM domesticCat Cat

Sentencia HQL	Equivalente en SQL
order by cat. name asc	ORDER BY Cat. name ASC
from Cat as cat	SELECT * FROM cat cat
inner join Cat. mate as mate	INNER JOIN cat_mate mate ON Cat. id = mate. id

A continuación, se presentan diferentes ejemplos haciendo uso de consultas por medio de HQL:

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.javahibernate.hibernate.demo.entity.Usuario;

public class ConsultarUsuarios {

    public static void main(String[] args) {

        // Crear el SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();
```

```
// Crear el Session

Session session = factory.getCurrentSession();

try {

    // Consultar todos los usuarios de la tabla Usuario

    List<Usuario> usuariosRegistrados = session.createQuery("from
Usuario").list();

    for (Usuario usuario : usuariosRegistrados) {

        System.out.println(usuario);

    }

    // Consultar usuarios con nombre que comienza con 'J'

    usuariosRegistrados = session.createQuery("from Usuario u where
u.nombre like 'J%'").list();

    for (Usuario usuario : usuariosRegistrados) {

        System.out.println(usuario);

    }

    // Consultar usuarios con apellido 'Ramirez'
```

```
usuariosRegistrados = session.createQuery("from Usuario u where
u.apellido = 'Ramirez']").list();

for (Usuario usuario : usuariosRegistrados) {

    System.out.println(usuario);

}

// Consultar usuarios con email que contiene 'javacode'

usuariosRegistrados = session.createQuery("from Usuario u where u.email
like '%javacode%'").list();

for (Usuario usuario : usuariosRegistrados) {

    System.out.println(usuario);

}

// Consultar usuarios con nombre 'Francisco' o que su password es
'12345'

usuariosRegistrados = session.createQuery("from Usuario u where
u.nombre = 'Francisco' OR u.password = '12345'").list();

for (Usuario usuario : usuariosRegistrados) {

    System.out.println(usuario);

}
```

```
    } finally {  
  
        factory.close();  
  
    }  
  
}  
  
}
```

Actualizaciones

El proceso de actualización es muy similar al proceso de consulta. Se puede hacer uso directo de los objetos obtenidos de las clases de entidad mediante los métodos modificadores de sus atributos (métodos set()) y confirmar las transacciones realizadas. También se puede hacer uso de createQuery(), el cual permite enviar consultas HQL un poco más elaboradas, pero asociándolo con el método executeUpdate() para que los cambios se reflejen en la base de datos una vez se confirma la transacción.

```
import java.util.List;  
  
import org.hibernate.Session;  
  
import org.hibernate.SessionFactory;  
  
import org.hibernate.cfg.Configuration;  
  
import com.javahibernate.demo.entity.Usuario;  
  
public class ConsultarUsuario {
```

```
public static void main(String[] args) {  
  
    // Crear el SessionFactory  
  
    SessionFactory factory = new Configuration()  
  
        .configure("hibernate.cfg.xml")  
  
        .addAnnotatedClass(Usuario.class)  
  
        .buildSessionFactory();  
  
  
    // Crear el Session  
  
    Session session = factory.getCurrentSession();  
  
  
    try {  
  
        // Iniciar una transacción  
  
        session.beginTransaction();  
  
  
  
        // Consultar un usuario por su clave primaria igual a 6  
  
        Usuario consultaUsuario = session.get(Usuario.class, 6);  
  
        consultaUsuario.setUserName("Nuevo Raul");  
  
        System.out.println("Usuario = " + consultaUsuario.getUserId() + "  
Actualizado");  
    }  
}
```

```
// Confirmar la transacción

session.getTransaction().commit();

System.out.println("Proceso finalizado");


    } finally {

        factory.close();

    }

}

}

import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.javahibernate.demo.entity.Usuario;

public class ActualizarUsuarios {
```

```
public static void main(String[] args) {  
  
    // Crear el SessionFactory  
  
    SessionFactory factory = new Configuration()  
  
        .configure("hibernate.cfg.xml")  
  
        .addAnnotatedClass(Usuario.class)  
  
        .buildSessionFactory();  
  
  
    // Crear el Session  
  
    Session session = factory.getCurrentSession();  
  
  
    try {  
  
        // Iniciar una transacción  
  
        session.beginTransaction();  
  
  
        // Actualizar la contraseña de todos los usuarios cuyo password actual sea  
'678'  
  
        System.out.println("Actualizando varios usuarios...");  
  
        session.createQuery("update Usuario u set u.userPassword = 'Z@Q#J!'  
where u.userPassword = '678'")
```

```
        .executeUpdate();

        // Confirmar la transacción
        session.getTransaction().commit();

        System.out.println("Proceso finalizado");

    } finally {

        factory.close();

    }

}
```

Método Delete

El método `delete()` del objeto `session` permite eliminar un registro de la base de datos mediante un objeto recibido como parámetro, que identifica el registro de la tabla a eliminar. La forma correcta de realizar un proceso de eliminación de un registro particular incluye los siguientes pasos:

- a) Iniciar una transacción por medio del método `beginTransaction()`.
- b) Utilizar un mecanismo para obtener una instancia de la clase entidad con el objeto que referencia el registro a eliminar, ya sea por medio del método `get()` o `createQuery()`.

- c) Invocar el método delete() del objeto session, pasando como parámetro el objeto que referencia el registro a eliminar.
- d) Confirmarla transacción realizada por medio del método getTransaction().commit().

```
import java.util.List;

import org.hibernate.Session;

import org.hibernate.SessionFactory;

import org.hibernate.cfg.Configuration;

import com.javahibernate.demo.entity.Usuario;

public class EliminarUsuarioDemo {

    public static void main(String[] args) {

        // Crear el SessionFactory

        SessionFactory factory = new Configuration()

            .configure("hibernate.cfg.xml")

            .addAnnotatedClass(Usuario.class)

            .buildSessionFactory();

        // Crear el Session

        Session session = factory.getCurrentSession();
```

```
try {  
  
    // Iniciar una transacción  
  
    session.beginTransaction();  
  
  
    // Eliminar un usuario cuyo id es 6  
  
    System.out.println("Eliminación de un usuario");  
  
    Usuario usuarioDelete = session.get(Usuario.class, 6);  
  
    session.delete(usuarioDelete);  
  
    System.out.println("Usuario " + usuarioDelete.getUserName() + "  
eliminado");  
  
  
    // Confirmar la transacción  
  
    session.getTransaction().commit();  
  
    System.out.println("Proceso finalizado");  
  
  
} finally {  
  
    factory.close();  
  
}
```

```
}  
  
}
```

```
import java.util.List;  
  
import org.hibernate.Session;  
  
import org.hibernate.SessionFactory;  
  
import org.hibernate.cfg.Configuration;  
  
import com.javahibernate.demo.entity.Usuario;  
  
  
public class EliminarUsuarioDemo {  
  
    public static void main(String[] args) {  
  
        // Crear el SessionFactory  
  
        SessionFactory factory = new Configuration()  
            .configure("hibernate.cfg.xml")  
            .addAnnotatedClass(Usuario.class)  
            .buildSessionFactory();  
  
  
        // Crear el Session
```

```
Session session = factory.getCurrentSession();

try {

    // Iniciar una transacción

    session.beginTransaction();

    // Eliminar todos los usuarios cuyo id es 6

    System.out.println("Eliminando el usuario de id = 6");

    session.createQuery("delete from Usuario u where u.userId =
6").executeUpdate();

    // Confirmar la transacción

    session.getTransaction().commit();

    System.out.println("Proceso finalizado");

} finally {

    factory.close();

}

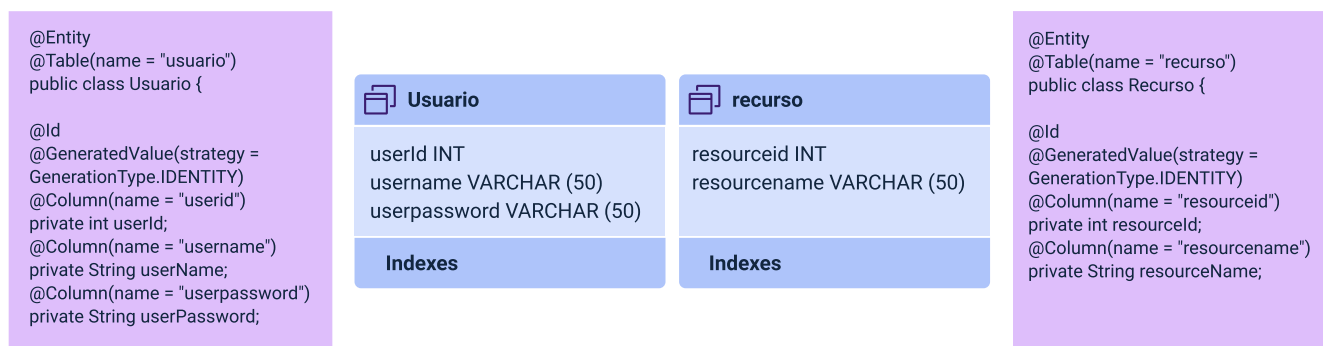
}
```

4.2. Asociaciones

Normalmente, al desarrollar una aplicación que gestione una base de datos, se requiere mucho más que una única tabla para la gestión de la información. Es común tener modelos de datos con múltiples tablas relacionadas con asociaciones de diferentes tipos y diferentes cardinalidades. Por lo tanto, es importante establecer en las clases de entidad este tipo de relaciones para que el proceso de mapeo sea coherente. Los tipos de asociaciones más comúnmente utilizadas son las que expresan relaciones uno a uno, relaciones uno a muchos, relaciones muchos a uno y relaciones muchos a muchos.

Para explicar cada uno de estos casos, utilizaremos como referencia el siguiente modelo de datos que ya fue mapeado de forma individualizada. A medida que se agregue una asociación de diferente tipo, se establecerán los cambios que deberían estar presentes, y que se detallan en la siguiente figura:

Figura 33. Asociaciones

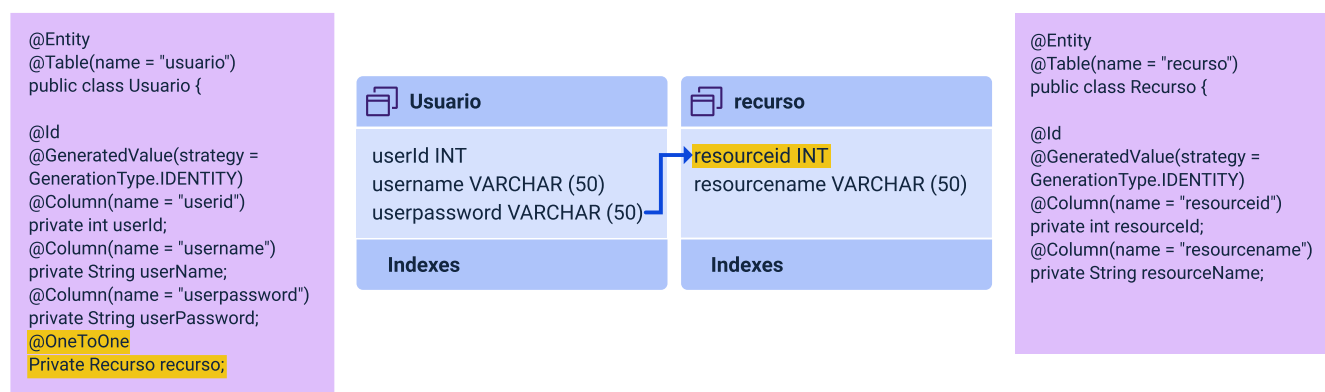


Relaciones OneToOne

En este tipo de relaciones, se quiere expresar un vínculo en el que un ejemplar de una tabla tiene relación con uno y solo un ejemplar de una segunda tabla. Desde el

punto de vista de las bases de datos, se creará una relación de integridad referencial vinculando una llave primaria con una llave foránea. Tomando como referencia el modelo de datos presentado anteriormente, supongamos que se desea indicar que un usuario tiene asociado un recurso. Para resolver este nuevo requerimiento, debemos modificar la clase de entidad de usuario para agregar un nuevo atributo que haga referencia al objeto recurso con el cual está relacionado y agregar la anotación correspondiente llamada OneToOne, de forma que Hibernate haga el mapeo correcto. El esquema se detalla a continuación en la siguiente figura:

Figura 34. Relaciones OneToOne

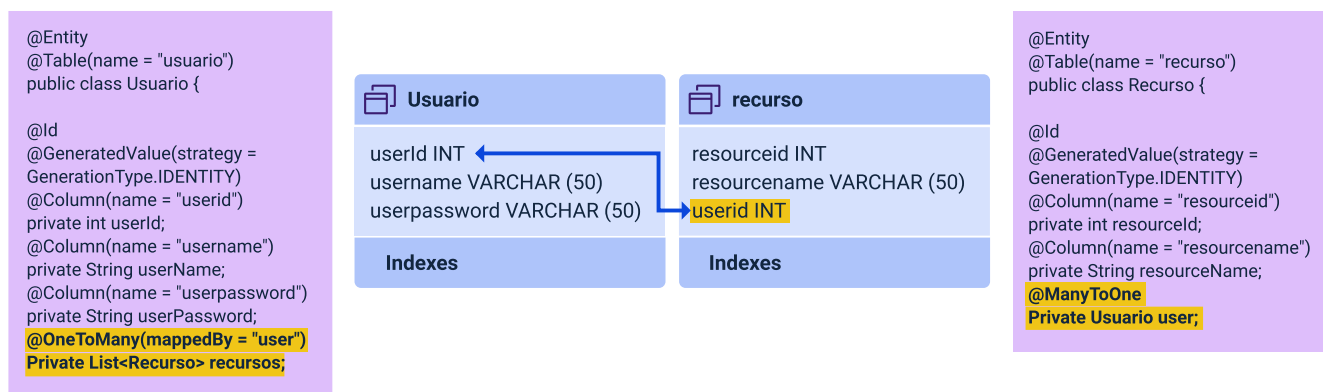


Mientras que en la base de datos se debe agregar un nuevo campo foráneo en la tabla usuario que haga referencia a la llave primaria de la tabla recurso para poder establecer la relación, en las clases de tipo entidad bastará con especificar el nuevo atributo del tipo de objeto correspondiente que referencia agregando la anotación de tipo OneToOne. Si el campo nuevo requerido en la tabla usuario no existe, el mapeador de Hibernate se encargará de agregarlo.

Relaciones OneToMany - ManyToOne

Las relaciones "OneToMany" tienen la particularidad de afectar cada una de las entidades vinculadas en la relación y es importante identificar en qué lugar es una relación "OneToMany" y en qué lugar la relación es "ManyToOne". Recordando las reglas de transformación del modelo lógico de una base de datos relacional, en este tipo de relaciones se indica que en la tabla del lado de la relación donde la cardinalidad máxima es muchos (*) deberá ser transferido como un campo foráneo la llave primaria de la tabla donde la cardinalidad máxima es 1, como se aprecia en la siguiente figura:

Figura 35. Relaciones OneToMany - ManyToOne



Para el caso de las clases de entidad, deberá especificarse en la clase donde la cardinalidad máxima es 1, un atributo del tipo lista de la clase de entidad con la que tiene un vínculo OneToMany. Este atributo será de tipo lista porque se está vinculando con varios. En el ejemplo, la entidad usuario se relaciona con varios recursos, es decir, un usuario puede estar vinculado con múltiples recursos.

Por otra parte, la clase de entidad donde la cardinalidad máxima es muchos deberá especificar un atributo de tipo objeto con la entidad vinculada, agregándole la anotación ManyToOne. En el ejemplo, un recurso estará siempre vinculado con un

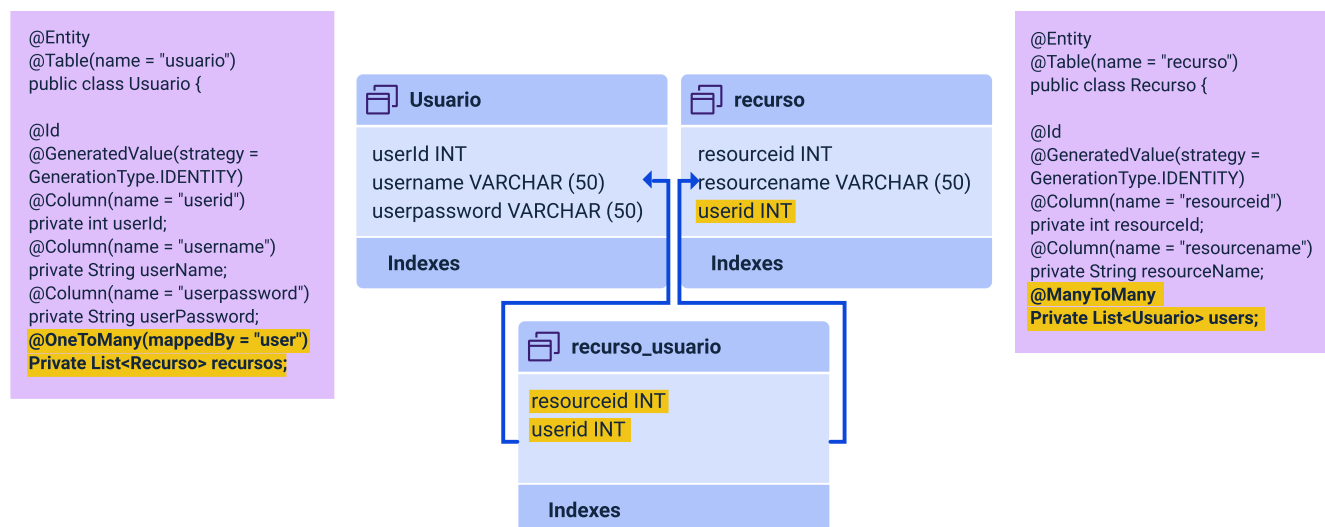
único usuario. Es importante anexar en la entidad de cardinalidad OneToMany el nombre del atributo en la entidad ManyToOne que será usado para el mapeo de la asociación por parte de Hibernate. En el ejemplo, se indica en la entidad Usuario que la relación OneToMany será mapeada usando el atributo user de la entidad ManyToOne.

Relaciones ManyToMany

Las relaciones ManyToMany obligan a la construcción de una nueva entidad de mapeo en la cual se alojarán las respectivas llaves primarias de las entidades vinculadas por medio de la relación ManyToMany. Para su correcta implementación, deberán agregarse en cada una de las clases entidad un objeto de tipo lista de la entidad a la cual está relacionada, etiquetada con la anotación ManyToMany. Adicionalmente, en una de las clases de entidad se deberá especificar el objeto de mapeo del otro extremo que será utilizado por Hibernate para realizar la asociación.

En el ejemplo siguiente, se establece una relación ManyToMany entre la clase Usuario y la clase Recurso, por lo cual en cada una de ellas se agrega la correspondiente lista del tipo de entidad a la cual se asocian con la anotación ManyToMany. En el caso de la clase Usuario, se agrega el atributo de tipo lista de Recursos llamado recursos para indicar que un usuario puede estar vinculado con muchos recursos. Por otro lado, en la clase Recurso se agrega una lista de tipo Usuario para indicar que un recurso puede estar vinculado con múltiples usuarios. Adicionalmente, la relación entre usuario y recurso se mapea con el objeto users presente en la clase Recurso. La relación entre recurso y usuario no se mapea explícitamente, por lo que generará la tabla intermedia recurso_usuario, lo cual cumple las reglas de transformación de un modelo lógico para una relación de muchos a muchos.

Figura 36. Relaciones ManyToMany



4.3. Ciclo de vida

Los objetos manipulados por Hibernate para el proceso de gestión de la persistencia pasa por cuatro estados diferentes, a continuación, se describe cada uno de ellos:

- **Estado transitorio**

Corresponde al primer estado de un objeto de entidad y sucede cuando instanciamos y manipulamos el objeto de tipo entidad sin conectarlo por medio de la sesión a un registro de la base de datos. Así, cualquier manipulación de este objeto no afecta la base de datos. En este estado, un objeto de tipo entidad se comporta como cualquier otro objeto JAVA.

- **Estado persistente**

Corresponde al estado que logra el objeto una vez se conecta con una sesión de Hibernate, lo cual ocurre cuando se invoca, por ejemplo, una operación de tipo save() o cuando el objeto fue producto de realizar un

proceso de consulta a la base de datos por un método `get()`. Todos los cambios que se realizan sobre un objeto en estado persistente son observados por Hibernate y podrán eventualmente realizar modificaciones en la base de datos.

- **Estado independiente**

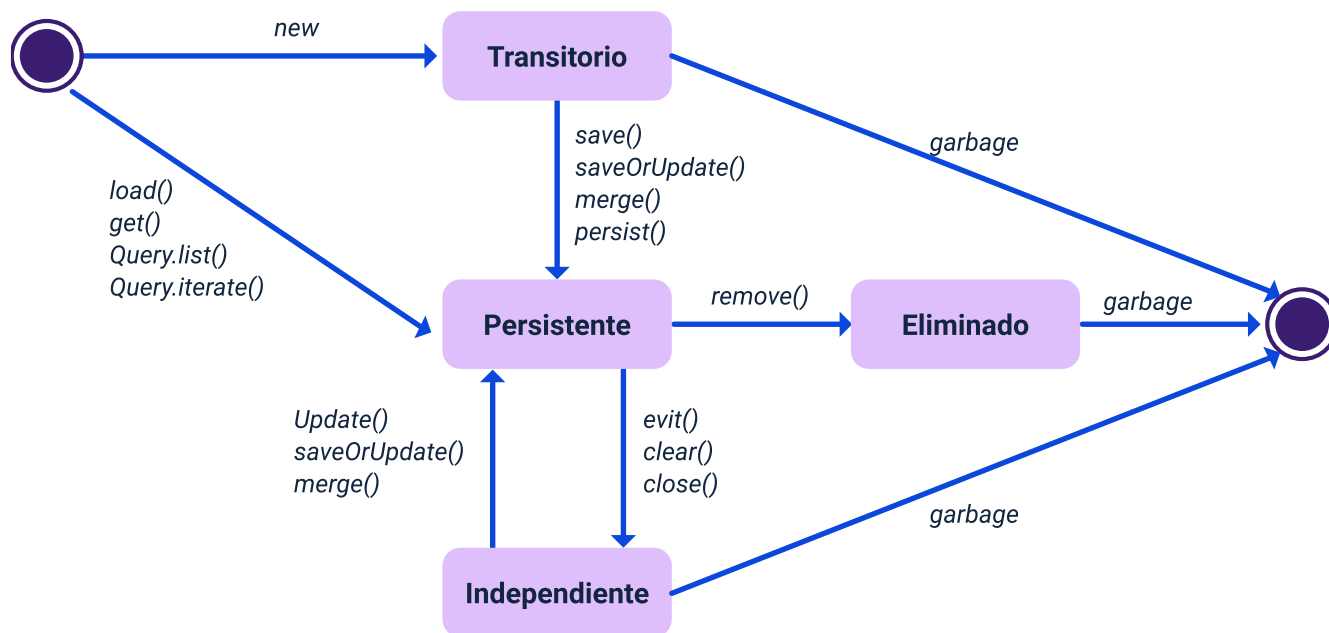
A este estado pasan los objetos en estado persistente una vez se realiza un cierre de sesión y en este momento son dejados de observar por Hibernate y cualquier cambio no será propagado en la base de datos. Un objeto independiente puede volver a ser persistente una vez se vuelva a abrir una nueva sesión y se use uno de los siguientes métodos: `save()` o `merge()`.

- **Estado eliminado**

Es el último estado en el ciclo de vida de un objeto de tipo entidad y ocurre cuando el objeto en cuestión es eliminado de la base de datos por medio del método `delete()`. Cualquier cambio realizado sobre este objeto no afectará la base de datos.

A continuación, se presenta un resumen de los estados del ciclo de vida y sus transiciones completas con todos los posibles métodos.

Figura 37. Ciclo de vida



5. Framework Spring

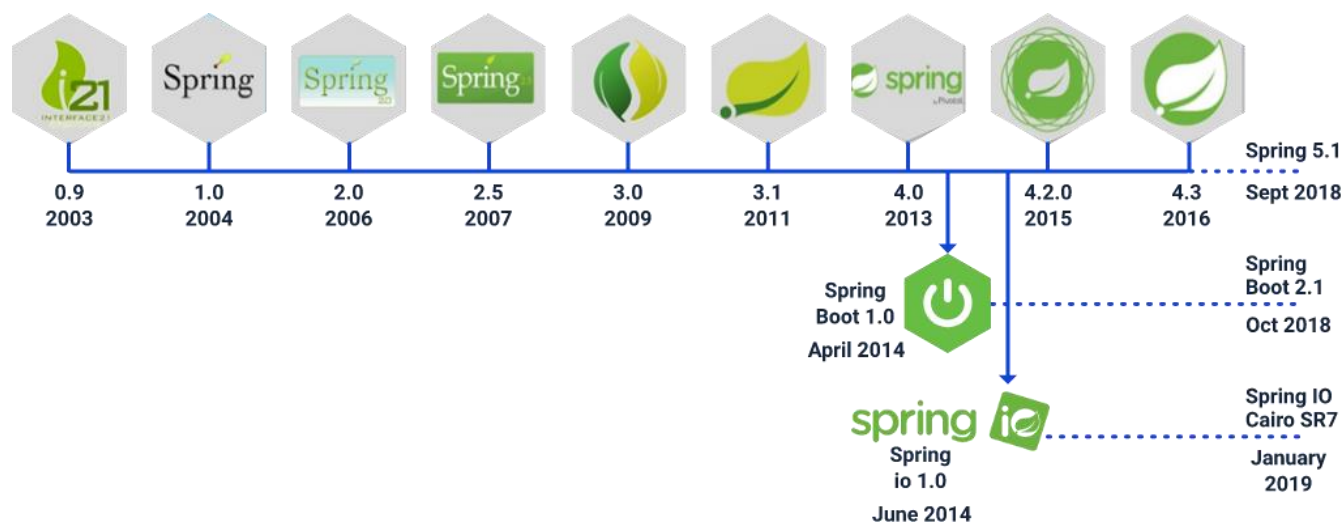
Spring es un framework de código abierto que facilita la creación de aplicaciones empresariales de alto rendimiento en Java. Además de Java, tiene soporte para los lenguajes Groovy y Kotlin. La estructura que lo compone es modular, ofreciendo flexibilidad para implementar una variedad de arquitecturas dependiendo de los requerimientos de las aplicaciones, haciendo que la programación sea lo más ágil posible, rápida y muy segura.

Historia

Rod Johnson, creador de Spring, se dio cuenta de que necesitaban una herramienta para simplificar y facilitar la construcción de aplicaciones JEE utilizando EJB. Escribió el libro *Expert One-on-One J2EE Development*, que es un conjunto de buenas prácticas para la creación de aplicaciones empresariales. En el año 2004, lanzó la primera versión 1.2.6 de Spring, marcando el inicio de J2EE tradicional (Johnson, 2004).

La siguiente figura presenta las versiones de Spring Framework:

Figura 38. Versiones de Spring Framework



A continuación, se presentan algunas de las características clave que hacen de Spring una opción popular entre los desarrolladores:

- **Flexibilidad**

Contiene un grupo flexible de librerías y extensiones de terceros de Spring, facilitando a los desarrolladores construir aplicaciones fácilmente.

- **Inyección de dependencia (DI)**

Ofrece un patrón que permite ayudar en el acoplamiento bajo. Esto significa que cuando existen dependencias entre las clases y se necesita crear un objeto de esta clase, Spring suministra este objeto, obteniendo beneficios en la modularidad, escalabilidad y dependencia de clases.

- **POJOs (Plain Old Java Objects)**

Facilita un sencillo desarrollo por medio de los POJOs.

- **ORM (Object Relational Mapping)**

Suministra un fácil acceso a datos por medio del ORM.

- **Estructura y módulos**

Spring integra una variedad de módulos que se pueden integrar para el desarrollo de aplicaciones.

A continuación, se detalla lo que se puede crear con Spring Framework: aplicaciones de consola, aplicaciones de escritorio, aplicaciones web, aplicaciones móviles, servicios SOAP y REST. Spring está orientado esencialmente al back-end.

- **Microservicios**

Permite crear y ejecutar microservicios en producción a escala.

- **Nube**

Ejecución de servicios en la nube

- **Aplicaciones web**

Creación rápida de aplicaciones web.

- **Reactive**

Permite la creación de proyectos reactivos basados en mensajes.

- **Servlets**

Permite la ejecución de código impulsado por eventos y se encarga del escalado la se tridad del lado del servidor.

- **Batch**

Aplicaciones de procesamiento de datos en lote.

5.1. Introducción

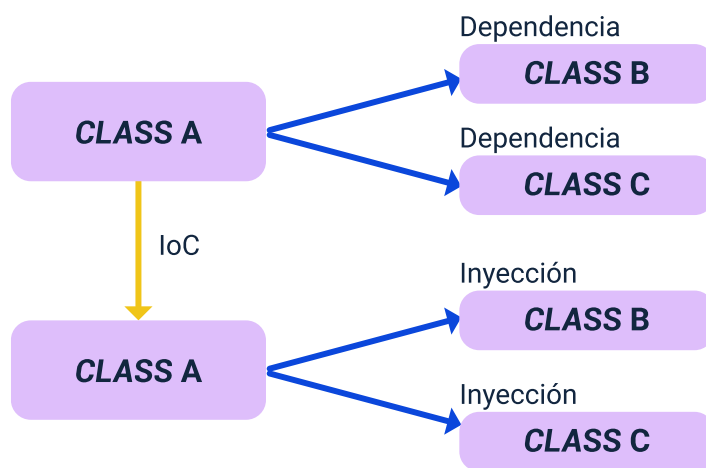
El framework Spring, básicamente con su inyección de dependencias, se encarga de construir objetos y configurar todas las dependencias que estos necesiten. En Spring, estos objetos creados son llamados beans, y se anotan en las clases de la

siguiente manera: @Component, @Service, @Repository, etc. Uno de los principios principales de Spring es el de Inversión de Dependencia (DIP), el cual establece que los módulos de alto nivel no deben depender de los de bajo nivel. Esto significa que se debe hacer referencia a las abstracciones más que a las implementaciones, evitando depender de una clase o implementación en particular.

La implementación de este principio se llama Inversión de Control (IoC), que se encarga de desacoplar el código utilizando eventos. Este principio también se conoce como el principio de Hollywood: "no nos llames; nosotros te llamaremos". Consiste en una manera de programar donde la aplicación interviene en lugar del código. Para implementar el IoC, se necesita un agente externo llamado contenedor, que es el encargado de verificar el flujo de la aplicación. Los IoC utilizan Service Locator, eventos y DI (inyecciones de dependencias), enfocando el Spring Framework en la DI.

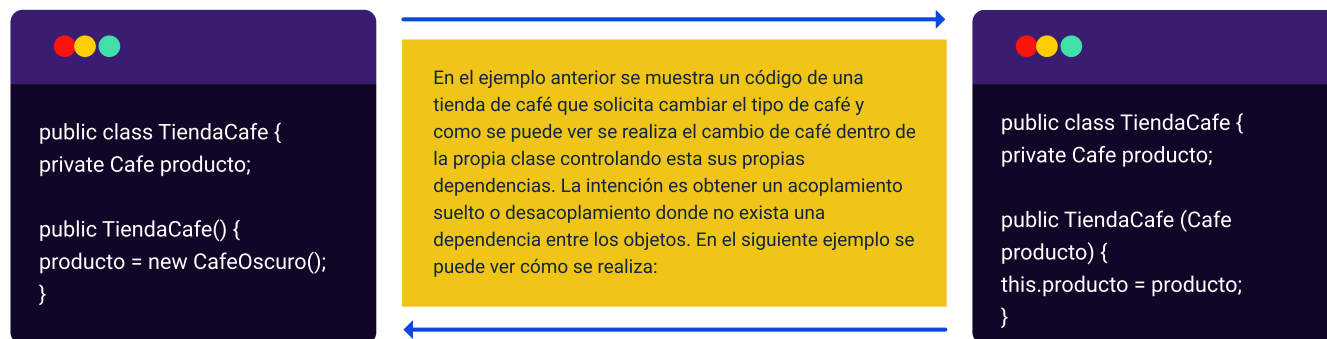
Inyección de dependencias: es un patrón de diseño de software, subtipo de inversión de control, cuyo objetivo es tener un código fácil de mantener o desacoplado. Su función es determinar lo que los objetos necesitan.

Figura 39. Principio inversión de controlguración



A continuación, se presenta un ejemplo de código donde los objetos dependen entre sí:

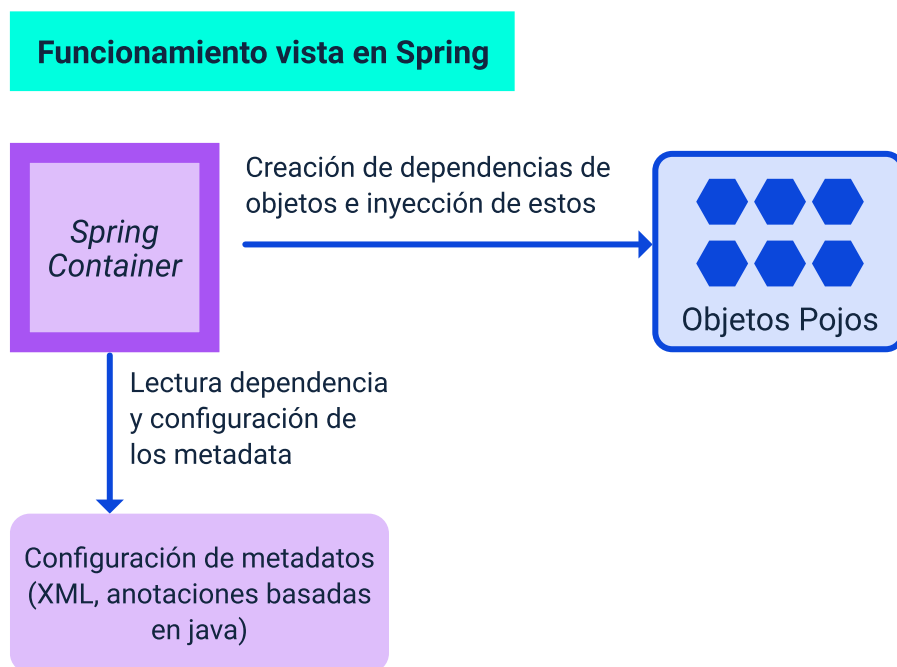
Figura 40. Acoplamiento hermético



En el código anterior, se obtuvo el objeto desde el exterior y que el objeto puede ser cualquier tipo de café, sin necesidad de modificar la clase `TiendaCafe`. Resumiendo, la inversión de control es un patrón de programación para obtener un acoplamiento flexible, logrando que los objetos independientes no sean responsables de sus dependencias, sino que las reciban desde afuera. El contenedor de IoC recoge los metadatos que vienen de un archivo XML o código Java y recibe las indicaciones de los objetos que debe crear, enlazar, configurar y destruir por medio de POJOs (Objetos simples de Java), creando objetos por medio del proceso que recibe el nombre de Spring Beans y resultando en objetos preparados para la inyección de dependencias.

La siguiente figura describe el funcionamiento de una vista abstracta de Spring.

Figura 41. Vista abstracta de Spring



Módulos del Framework

Spring Framework es un marco de trabajo muy amplio y brinda cerca de 20 módulos, los cuales se asocian en el contenedor principal: accesos e integración de datos, web, AOP (Programación orientada a aspectos), instrumentación, mensajes y pruebas (Spring, 2021).

Para trabajar con Spring se necesita tener instalados los siguientes requerimientos:

- **Un entorno de desarrollo (IDE).**

Se puede seleccionar cualquiera de los relacionados en la tabla No 6.

- **Un JDK (Kit de desarrollo de Java).**

Se puede descargar cualquiera de los relacionados en la tabla No 7.

Tabla 6. Entornos para desarrollo en Spring

Entornos de desarrollo	Enlace
IntelliJ IDEA	https://www.jetbrains.com/idea/
Spring Tools	https://spring.io/tools
Visual Studio Code	https://code.visualstudio.com/docs/languages/java
Eclipse	https://www.eclipse.org/downloads/packages/
NetBeans	https://netbeans.apache.org/

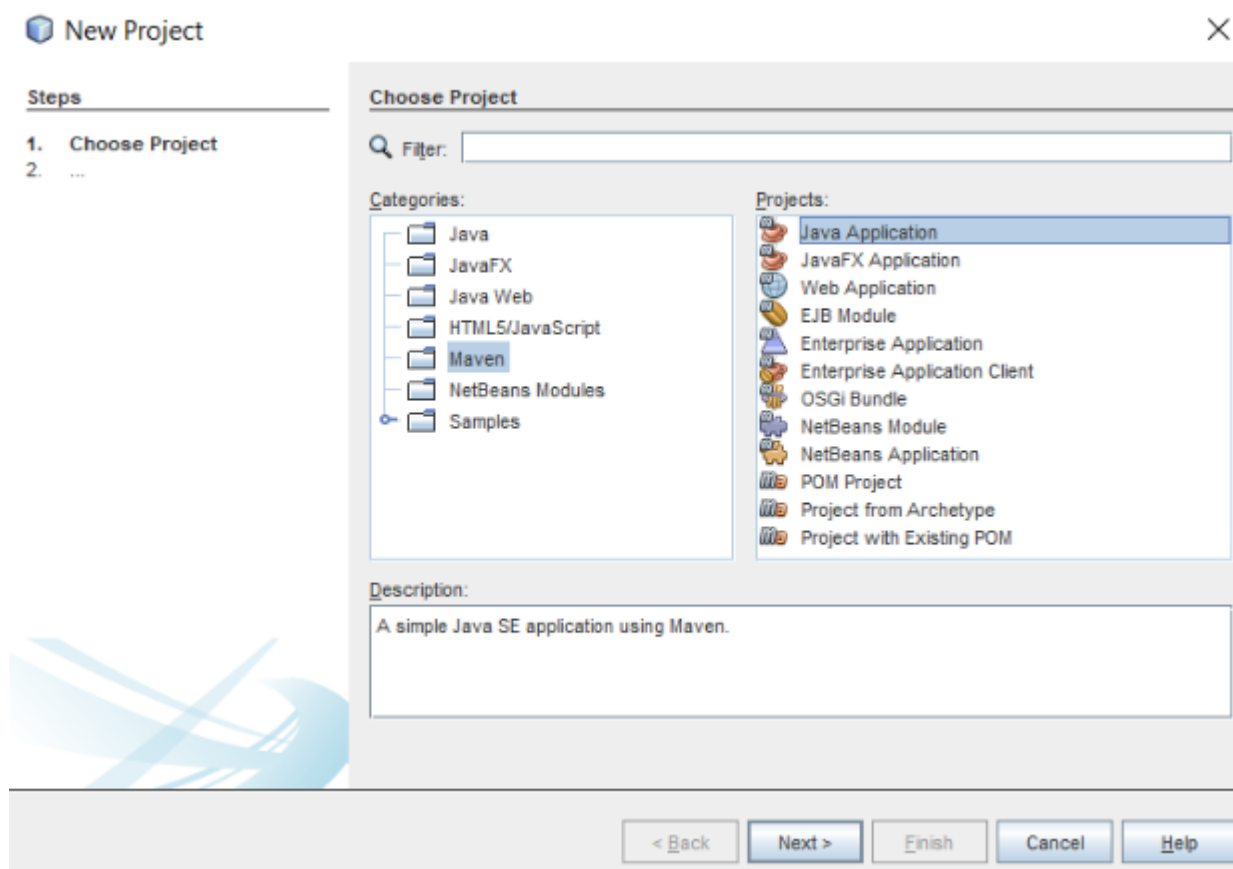
Tabla 7. JDK para desarrollo en Spring

JDK	Enlace
JDK Oracle	https://www.oracle.com/java/technologies/downloads/
AdoptOpenJDK	https://adoptopenjdk.net/

Para construir aplicaciones en Spring Framework necesitamos:

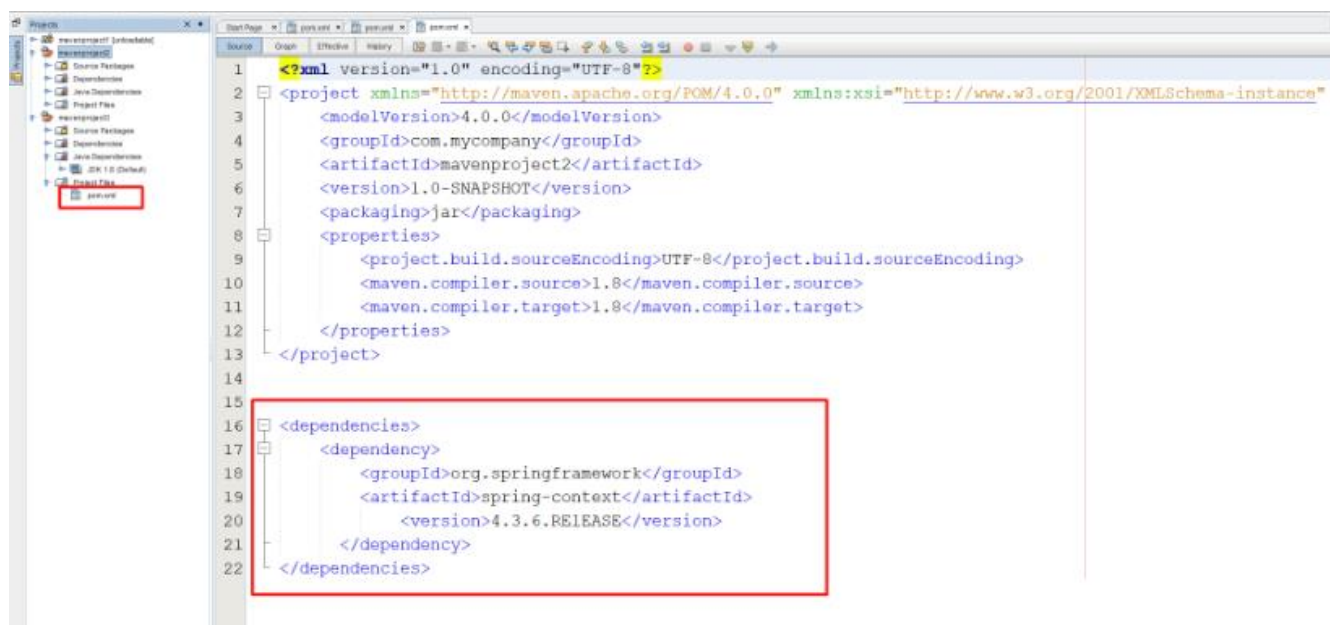
- Elegir Jars con Maven: lo primero que se debe hacer es crear un proyecto Maven o Gradle y realizar la descarga de las dependencias necesarias.

Figura 42. Ventana de creación de un nuevo proyecto en un IDE



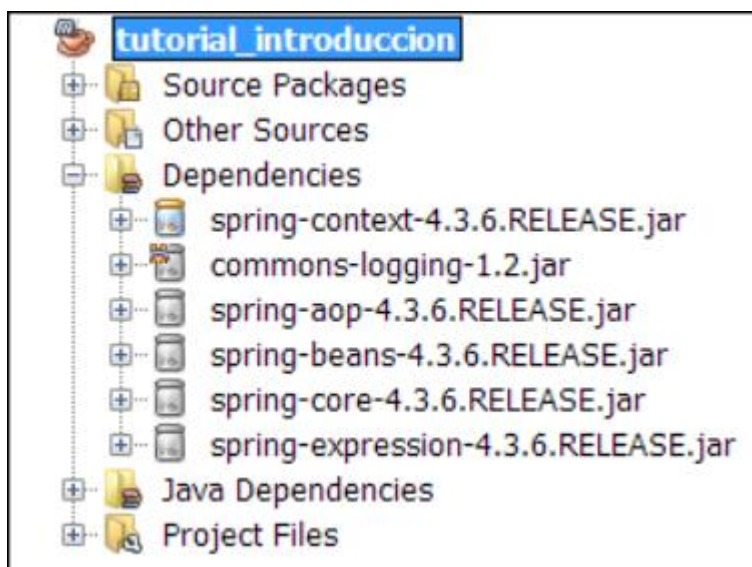
- Configuración de las dependencias en el archivo pom.xml.

Figura 43. Pantalla de un archivo XML de Maven



- Las dependencias se deben listar en la carpeta Dependencias.

Figura 44.



Creación de la aplicación

A continuación, se presenta como crear la aplicación:

- **Creación de la Aplicación**

Para el ejemplo se creará un programa con el ejemplo de HolaPlaneta.

```
public interface HolaPlaneta {  
  
    void enviarSaludo ();  
  
}
```

```
public class HolaPlanetaImpl implements HolaPlaneta {  
  
    @Override  
  
    public void enviarSaludo () {  
  
        System.out.println("Hola Planeta estamos en Spring Framework");  
  
    }  
  
}
```

- **Configurar archivo XML**

Una vez se tenga la aplicación, se realizará la configuración en el archivo springXMLconfig.

```
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="http://www.springframework.org/schema/beans  
        http://www.springframework.org/schema/beans/spring-beans-4.3.xsd">  
  
    <bean id="saludaService" class="sena.spring.introduccion.HelloPlanetaImpl"/>  
  
</beans>
```

```
</beans>
```

- **Ejemplo de código**

A continuación, se presenta un nuevo ejemplo de código explicando más a detalle el uso del Contenedor en Spring. Para el siguiente ejemplo se tiene un proyecto llamado “Sena” el cual tiene una clase Aprendiz con los siguientes atributos:

```
package com.sena
```

```
public class AprendizBean {
```

```
    private String primernombre;
```

```
    private String apellidos;
```

```
    private String cedula;
```

```
    public String getPrimerNombre () {
```

```
        return primernombre;
```

```
    }
```

```
    public void setPrimerNombre (String nombre) {
```

```
        this.primernombre = nombre;
```

```
}
```

```
public String getApellidos () {
```

```
    return apellidos;
```

```
}
```

```
public void setApellidos (String apellidos) {
```

```
    this.apellido = apellidos;
```

```
}
```

```
public String getCedula () {
```

```
    return cedula;
```

```
}
```

```
public void setCedula (String cedula) {
```

```
    this.cedula = cedula;
```

```
}
```

```
public void verAprendiz () {
```

```
System.out.println(primernombre+" "+apellidos+", es la actual estudiante");  
  
}
```

```
}
```

- **Gestión de objetos**

Al utilizar este proyecto Sena, Spring framework cuenta con un archivo XML que gestiona objetos (Spring IOC Container). Para este proyecto, recibe el nombre de springconfig.xml y se ve de la siguiente manera.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>  
<beans xmlns="http://www.springframework.org/dtd/spring-beans-2.0.dtd" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://www.springframework.org/dtd/spring-beans-2.0.dtd http://www.springframework.org/dtd/spring-beans-2.0.dtd">  
  <bean id="aprendiz" class="com.sena.AprendizBean">  
    <property name="primernombre" value="Zulema">  
    <property name="apellidos" value="Leon">  
    <property name="cedula" value="25.311.165">  
  </bean>  
</beans>
```

- **Configuración del archivo**

En la configuración del archivo, se ve que la etiqueta bean referencia la clase AprendizBean.java. Además, el atributo class dirige a la dirección del proyecto y

cuenta con un atributo id que permite agregar un nombre para poder identificarlo. Las demás etiquetas como property indican los atributos de la clase AprendizBean y con la etiqueta value se pueden agregar valores directos. Por medio de los métodos get y set, el Spring Container obtiene acceso y puede modificar los objetos de la clase AprendizBean. Con los archivos de configuración XML, Spring facilita una clase de listas por cada uno de los beans existentes para poder hacer llamado de ellos y así poder manipularlos desde otras clases sin tener que realizar varias instancias.

```
package com.sena
```

```
import org. springframework.beans.factory.BeanFactory;
```

```
import org.springframework.beans.factory.xml.XmlBeanFactory;
```

```
import org.springframework.core.io.ClassPathResource;
```

```
import org.springframework.core.io.Resource;
```

```
public class SenaLogic{
```

```
    public static void main(String[ ] args){
```

```
        Resource recurso = new ClassPathResource("springconfig.xml");
```

```
        BeanFactory factory = new XmlBeanFactory(recurso);
```

```
AprendizBean aprendizBean = (AprendizBean)factory.getBean("aprendiz");
```

```
aprendizBean.verAprendiz();
```

```
}
```

```
}
```

- **Implementación de Spring**

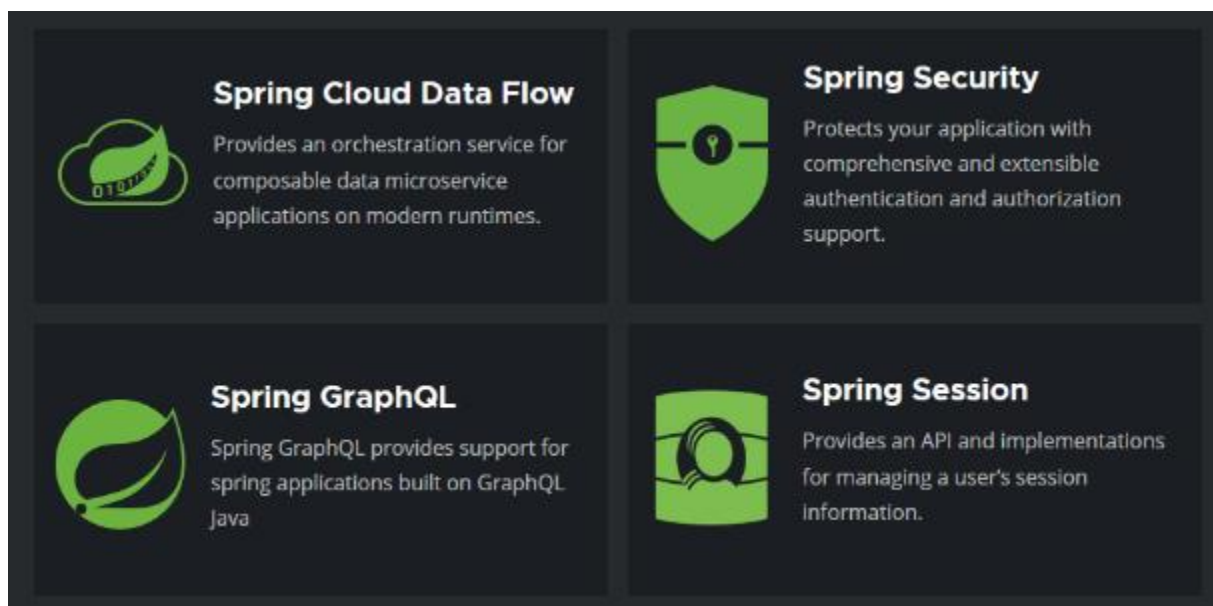
En el siguiente código se presenta la implementación de Spring del control de los objetos del proyecto Sena. La salida que muestra se logró realizando las configuraciones en el archivo XML. La utilidad del uso de Spring se evidencia en proyectos grandes con varias clases y varios objetos ya que facilita un manejo mucho más eficiente.

Zulema Leon, es la actual estudiante

5.2. Spring Data

Spring ofrece una variedad de proyectos independientes que se pueden articular en un proyecto principal. Cada proyecto permite resolver una necesidad especial del negocio; algunos de ellos se ilustran en la siguiente figura.

Figura 45. Spring



Uno de los proyectos más utilizados es Spring Data, ya que es uno de los módulos más grandes de Spring debido a que cuenta con muchos submódulos. Su objetivo principal es proporcionar una serie de herramientas que permiten realizar integraciones muy sencillas con los diferentes entornos de almacenes de datos disponibles. Los datos que se utilizan pueden proceder de orígenes tanto de bases de datos relacionales como no relacionales.

Las funcionalidades de Spring Data son:

- Poderosos repositorios y uso personalizado de mapeo de objetos.
- Creación de consultas dinámicas fundamentadas en los nombres que tienen los métodos.
- Plantillas de clases que brindan propiedades elementales que hacen más fácil utilizar los repositorios.

- Campos que hacen referencia a fecha de creación y modificación de las clases.
- Sencilla integración con Spring Framework por medio de la configuración de clase o archivos XML.

Módulos de Spring Data

Los módulos oficiales de Spring Data son:

- **Spring Data Commons.**
Brinda infraestructura compartida para módulos de Spring.
- **Spring Data JDBC.**
Implementa repositorios fundamentados en JDBC.
- **Spring Data JPA.**
Implementa repositorios fundamentados en JPA.
- **Spring Data Gemfire.**
Implementa GemFire para aplicaciones baja latencia.
- **Spring Data KeyValue.**
Implementa repositorios para datos de clave-valor.
- **Spring Data LDAP.**
Implementa repositorios para soporte de LDAP.
- **Spring Data MongoDB.**
Soporte para documentos y repositorios de MongoDB.
- **Spring Data Rest.**
Implementa repositorios Restful.

- **Spring Data Redis.**

Fácil acceso a Redis desde Spring.

- **Spring Data para Apache Cassandra.**

Implementación para aplicaciones orientadas a datos.

- **Spring Data para Apache Geode.**

Implementación para aplicaciones orientadas a datos.

A continuación, se presenta un ejemplo de un mapeo para JPA, que es uno de los submódulos de Spring Data. En el ejemplo, se desarrolla la clase Persona, la cual representa una tabla en la base de datos llamada Persona:

- a) La creación de la tabla se especifica mediante la anotación `@Entity` y en el atributo `name` se coloca el nombre de la tabla.
- b) Por medio de la anotación `@Column` se asigna cada uno de los campos de la tabla de la base de datos a los atributos de la clase Persona con la etiqueta "name" se hace referencia al nombre de la columna de la tabla.

```
@Entity(name = "persona")
```

```
public class Persona {
```

```
    @Column(name = "id")
```

```
    private Long id;
```

```
    @Column(name = "persona_nombre")
```

```
private String nombre;  
  
@Column(name = "persona_apellido")  
  
private String apellido;  
  
@Column(name = "persona_telefono")  
  
private String telefono;  
  
}
```

Al extender de la clase CrudRepository, se facilita el uso de métodos como findByNombre, que genera una consulta de todas las personas con el parámetro nombre. En bases de datos relacionales, esta operación se traduce en SELECT * FROM persona WHERE nombre=?. Este enfoque reduce considerablemente las sentencias SQL, permitiendo obtener consultas dinámicas de forma más eficiente.

5.3. Spring Boot y MySQL

Spring Boot es una infraestructura liviana que suprime en gran parte la tarea de configurar las aplicaciones fundamentadas en Spring. A diferencia de Spring Framework, Spring Boot realiza una simplificación en cuanto a configuración y el despliegue de las aplicaciones.

Las características de Spring Boot son:

- Es intuitivo: permite crear aplicaciones rápidamente gracias a sus valores predeterminados.

- Permite crear aplicaciones que son independientes de Spring.
- Incluye los servidores de Undertow, Tomcat o Jetty.
- Usualmente realiza la configuración automática de librerías de Spring o de otros.
- No requiere configuraciones de archivos XML.
- Agiliza el desarrollo.

Spring Boot cuenta con dos mecanismos que permiten la simplificación de los procesos: el contenedor de aplicaciones integrado y los starters.

Contenedor de aplicaciones integrado

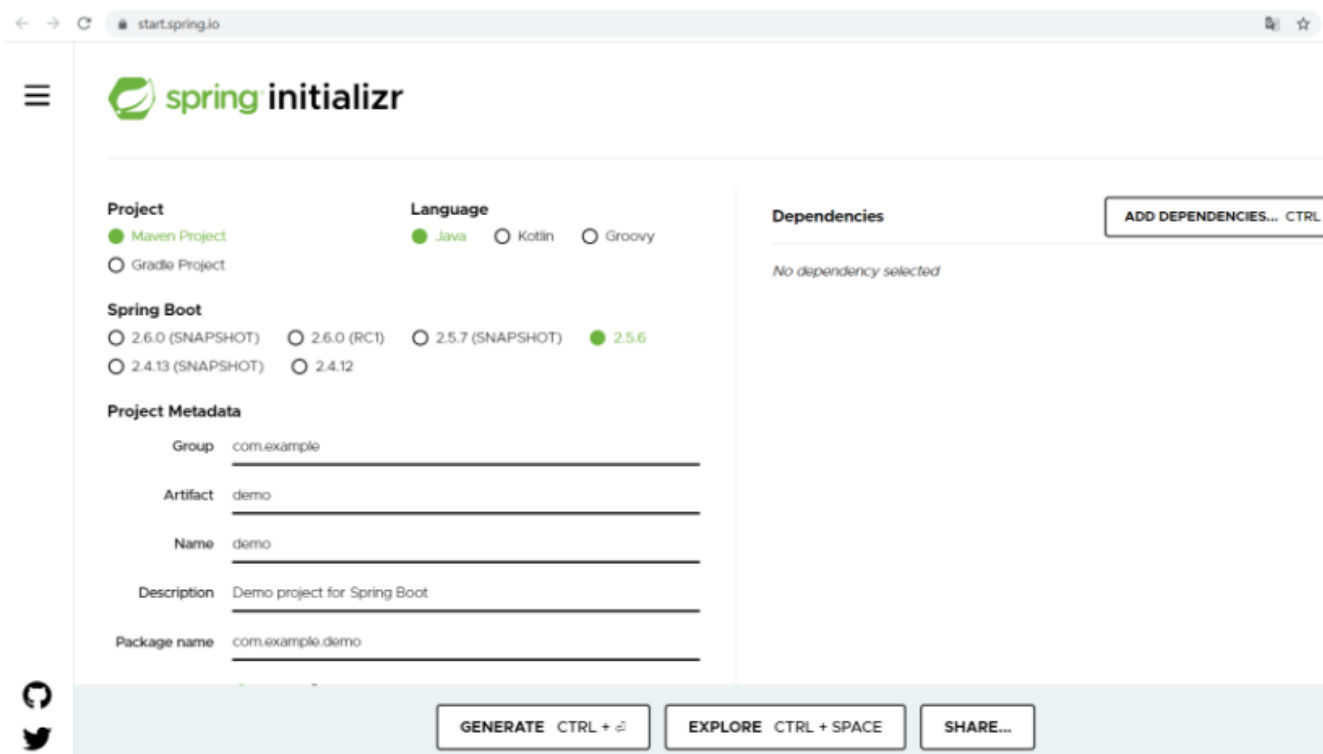
La compilación de las aplicaciones web se realiza como un archivo .jar desplegado en un servidor Tomcat, facilitando la distribución de las aplicaciones. Anteriormente, se trabajaba con .war, que si se requiere, se puede seguir utilizando.

Starters

Son los encargados de brindar una variedad de dependencias que se agregan al proyecto; estas pueden ser de terceros o de Spring. Los starters por defecto traen una configuración de valores predeterminados.

Otra de las ventajas de Spring Boot es que ofrece un mecanismo que genera la estructura del proyecto automáticamente. Este inicializador crea el archivo pom.xml, el main de la aplicación y un archivo de test para verificar que la aplicación funcione correctamente.

Figura 46. Inicializador de Spring



The screenshot shows the Spring Initializr web application at start.spring.io. The interface is divided into several sections:

- Project:** Includes radio buttons for **Maven Project** (selected) and **Gradle Project**.
- Language:** Includes radio buttons for **Java** (selected), **Kotlin**, and **Groovy**.
- Spring Boot:** Includes radio buttons for various versions: 2.6.0 (SNAPSHOT), 2.6.0 (RC1), 2.5.7 (SNAPSHOT), **2.5.6** (selected), 2.4.13 (SNAPSHOT), and 2.4.12.
- Project Metadata:** A form with the following fields:
 - Group:** `com.example`
 - Artifact:** `demo`
 - Name:** `demo`
 - Description:** `Demo project for Spring Boot`
 - Package name:** `com.example.demo`
- Dependencies:** A section with a button **ADD DEPENDENCIES... CTRL +** and the text *No dependency selected*.

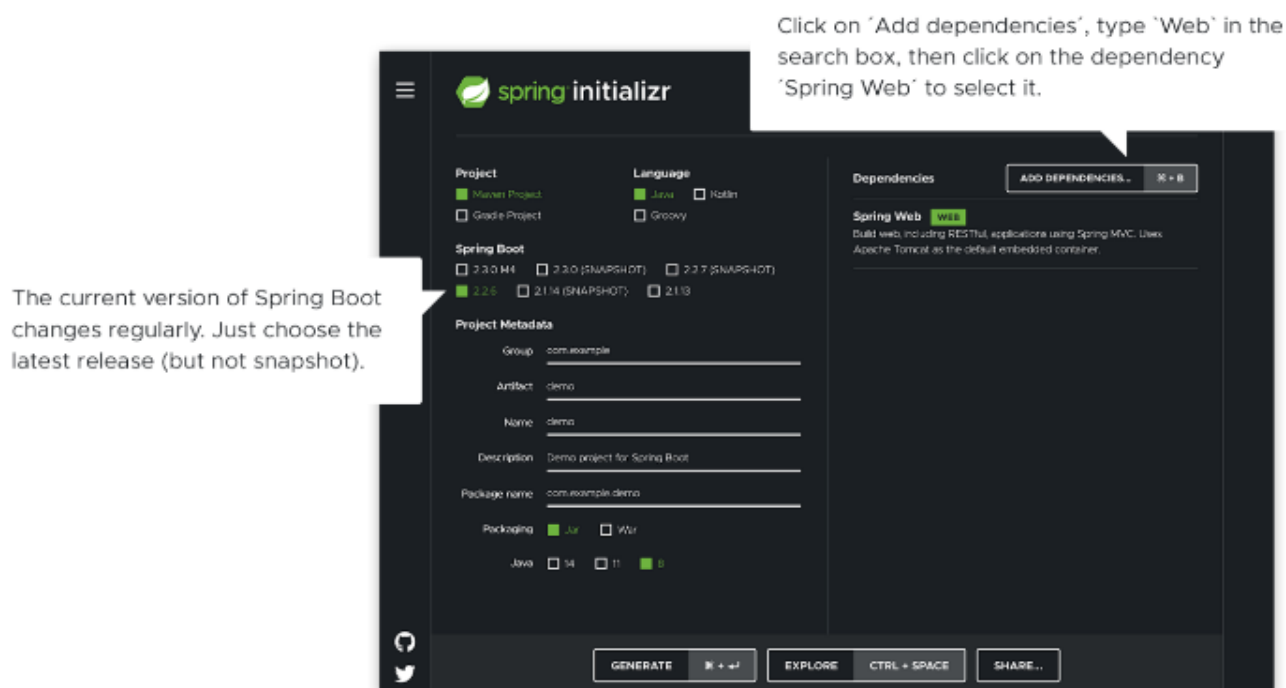
At the bottom, there are three buttons: **GENERATE CTRL + G**, **EXPLORE CTRL + SPACE**, and **SHARE...**.

A continuación, se realizan los pasos para la creación de un proyecto en Spring Boot. Hay que recordar que se deben tener instaladas cualquiera de las herramientas de Entorno de desarrollo y JDK.

- **Comienza un nuevo proyecto**

Ingresar a start.spring.io y crear un proyecto web.

Figura 47. Spring Initializr



- **Ingresar el código**

Después de descargar el archivo que genera el inicializador, ejecute su IDE y abra el archivo. Una vez ubicado en la clase principal, copie el siguiente código para ejecutar un Hola mundo. Lo que hace el método hello() es recibir un parámetro de tipo string con el nombre de name y después concatenar este parámetro con la palabra "Hello". Esto quiere decir que si se ingresa un nombre "Zulema", la salida sería "Hello Zulema".

Figura 48. Fragmento de código en Java para una aplicación Spring Boot

```
package com.example.demo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@SpringBootApplication
@RestController
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }

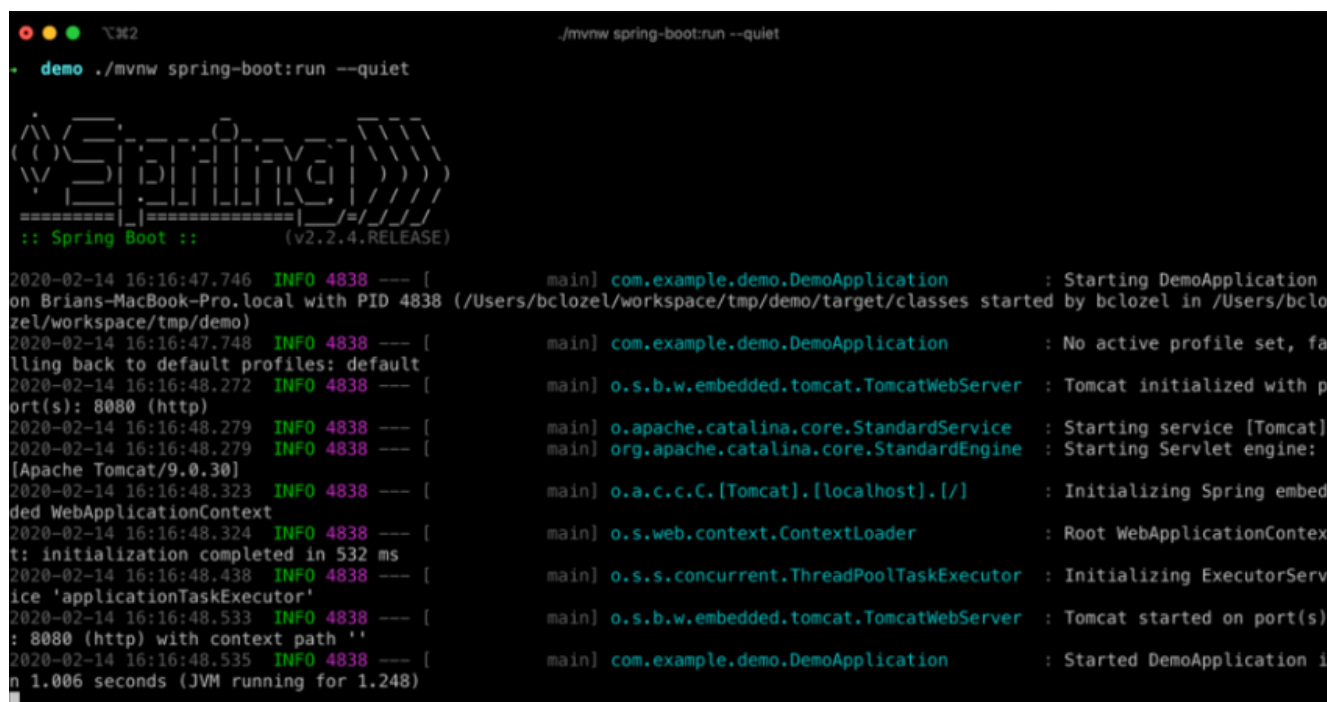
    @GetMapping("/hello")
    public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
        return String.format("Hello %s!", name);
    }
}
```

- **Prueba del proyecto**

Ahora solo es probar el proyecto. Para esto, abra una consola (Windows + R), ubíquese en la carpeta del proyecto y ejecute el siguiente comando:

```
mvn spring-boot:run
```

Figura 49. Salida de terminal



Lo que hace es ejecutar el servidor Tomcat que ya viene por defecto dentro de Spring Boot. Ahora se debe ingresar al navegador y escribir "localhost" con el puerto 8080, y se presentará la ejecución de la aplicación. En la siguiente videoclase se explica de forma más detallada cómo utilizar Spring Boot:

Video 6. Spring boot



[Enlace de reproducción del video](#)

Síntesis del video: Spring boot

En el video "Spring Boot", se explica cómo crear una aplicación sencilla utilizando Spring Boot. El proceso se detalla desde la instalación de las herramientas necesarias hasta la ejecución de un proyecto básico.

Primero, se mencionan los entornos de desarrollo compatibles, como IntelliJ, Spring Tools, Visual Studio Code, Eclipse y NetBeans. También se recomienda instalar el JDK de Java, preferiblemente OpenJDK. El tutorial utiliza Spring Tools para este ejemplo.

Una vez descargado e instalado Spring Tools, se crea un nuevo proyecto seleccionando "Spring Starter Project". Se nombra el proyecto y se elige Maven como gestor de dependencias. Luego, se agrega la dependencia "Spring Web" para desarrollar un servicio web.

El proyecto genera automáticamente un archivo pom.xml con todas las dependencias necesarias. Spring Boot integra un servidor web (Tomcat), eliminando la necesidad de configuraciones adicionales.

En el código del proyecto, se crea un método llamado Hello que imprime un saludo. Se usan anotaciones de Spring Boot para definir un controlador REST. Una vez configurado, el proyecto se ejecuta mediante clic derecho y seleccionando "Run as Spring Boot App".

Finalmente, se accede al navegador web, se ingresa localhost/hello, y se obtiene el saludo "Hola mundo". El video destaca cómo Spring Boot facilita el desarrollo enfocado en la lógica del negocio sin preocuparse por configuraciones complejas.

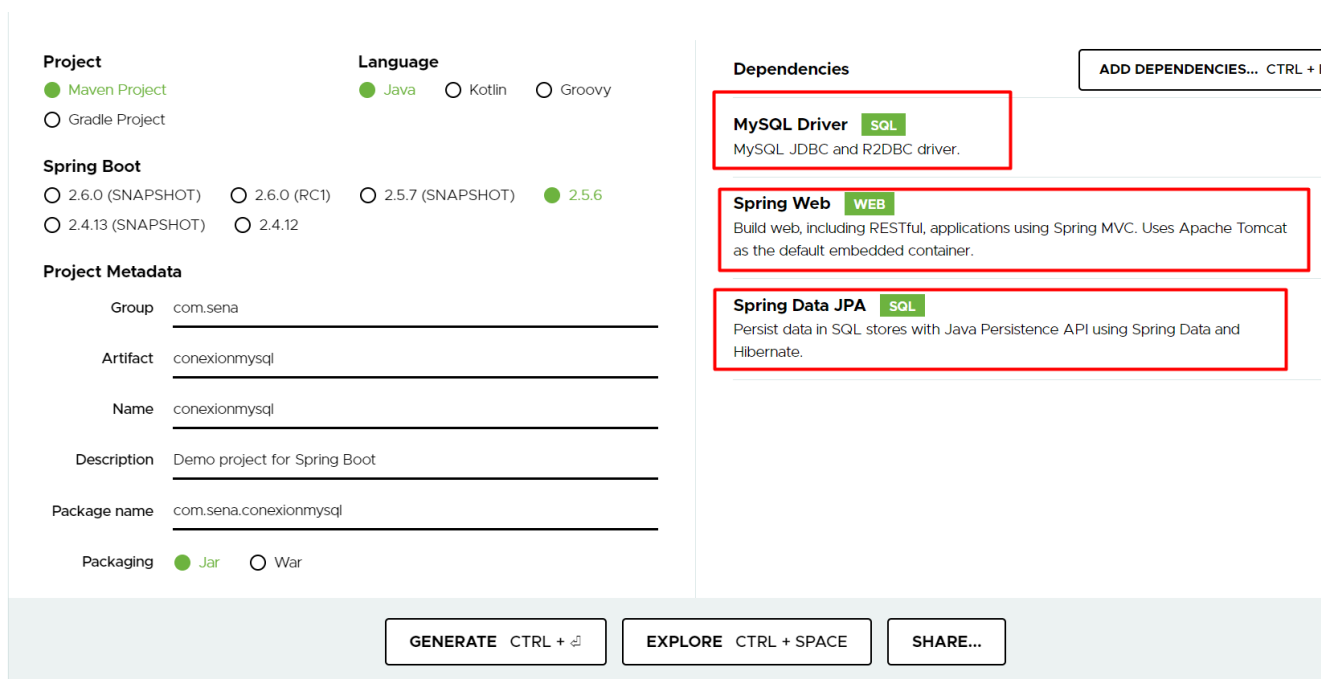
Spring Boot y MySQL

Para tener una aplicación web completa, es necesario almacenar información. Por esta razón, es importante realizar conexiones con bases de datos. En este ejemplo, se detalla cómo realizar la conexión con la base de datos MySQL.

Los requerimientos fundamentales son tener instalado MySQL, el entorno de desarrollo y seguir estos pasos:

- **Paso 1.** Declarar dependencias. JPA, MySQL JDBC Driver, Web. Como se mostró en el ejemplo anterior, por medio del inicializador se pueden seleccionar las dependencias requeridas, como se muestra en la siguiente imagen.

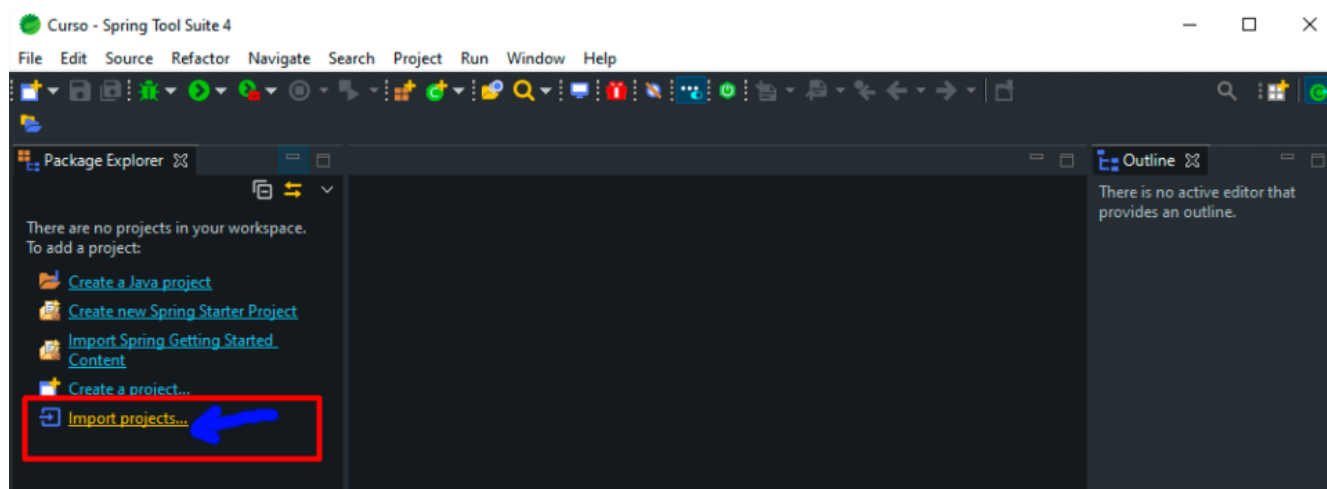
Figura 50. Vista del IDE Eclipse



The screenshot shows the Spring Initializr web interface. On the left, under 'Project', 'Maven Project' is selected. Under 'Language', 'Java' is selected. Under 'Spring Boot', version '2.5.6' is selected. The 'Project Metadata' section contains the following fields: Group (com.sena), Artifact (conexionmysql), Name (conexionmysql), Description (Demo project for Spring Boot), Package name (com.sena.conexionmysql), and Packaging (.Jar). On the right, the 'Dependencies' section shows three selected dependencies: 'MySQL Driver' (SQL), 'Spring Web' (WEB), and 'Spring Data JPA' (SQL). At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and 'SHARE...'.

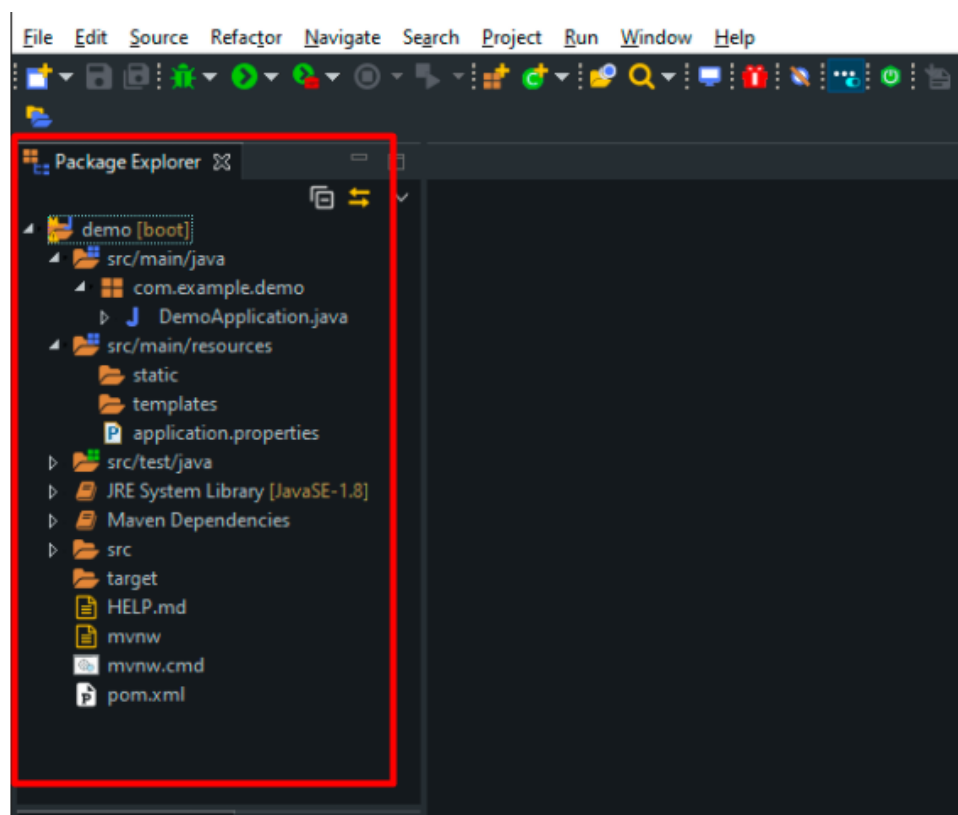
- **Paso 2.** Después de agregar las dependencias, clic en GENERATE y descargará un archivo zip el cuál se debe descomprimir y después importar el proyecto en el IDE, en este caso Spring Tool Suite.

Figura 51. Captura de pantalla del Spring Initializr



- **Paso 3.** Importar el proyecto. Una vez importado el proyecto, se puede ver cómo queda la estructura.

Figura 52. Vista del "Package Explorer" en Eclipse



El paso siguiente es abrir el archivo “application.properties” y configurar todas las propiedades de la conexión así:

```
spring.jpa.hibernate.ddl-auto=update
```

```
spring.datasource.url=jdbc:mysql://localhost:3306/demo?useUnicode=true&characterEncoding=utf8&createDatabaseIfNotExist=true
```

```
spring.datasource.username=root
```

```
spring.datasource.password=password
```

```
spring.jpa.show-sql=true
```

A continuación, se explicará cada línea del anterior código que va en el archivo “application. properties”.

Línea 1: permite obtener los metadatos de la base de datos.

Línea 2: se muestra la url de la conexión con el motor de base de datos.

Línea 3: Se especifica el nombre del usuario de la base de datos para la conexión

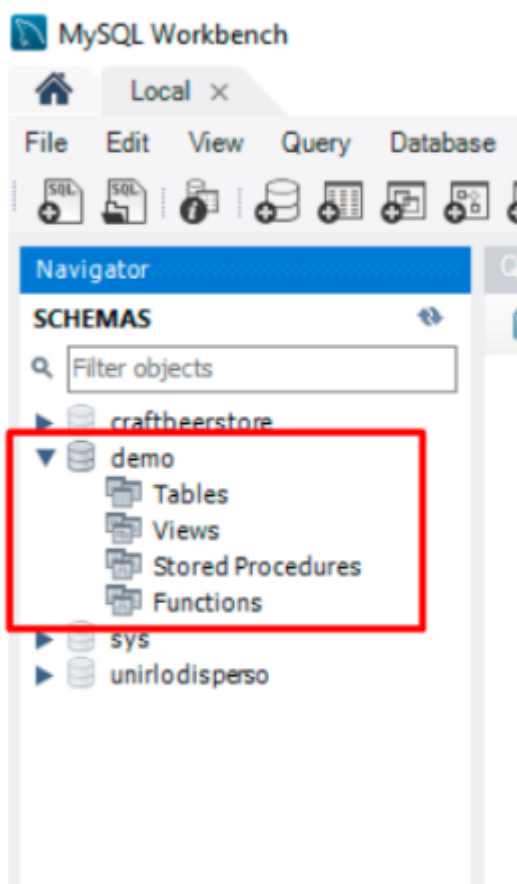
Línea 4: Se especifica clave de la base de datos para la conexión

Línea 5: la propiedad true permite utilizar las consultas JPA a realizar en la base de datos.

Lo que queda es ejecutar la aplicación y comprobar que no existan problemas con la conexión a la base de datos. Para ejecutar, dar clic derecho en el archivo Java, seleccionar la opción >Run As y >Spring Boot App.

Si todo está bien, ahora solo queda ir al motor de bases de datos y verificar si se creó el esquema.

Figura 53. Ejecución de la aplicación



5.4. Validaciones

Es muy importante realizar validaciones cuando se están desarrollando aplicaciones web en Java para impedir que se ingresen datos que afecten al programa. En Spring viene incluido el hibernate-validator en el paquete spring-boot-starter-web, que permite validar valores de los campos beans.

```
<dependency>
```

```
  <groupId>org.springframework.boot</groupId>
```

```
  <artifactId>spring-boot-starter-web</artifactId>
```

```
<version>2.0.0.RELEASE</version>
```

```
</dependency>
```

Para el ejemplo, se realiza la creación de un bean con unas anotaciones que permiten la validación de ingreso de un usuario.

```
@Data
```

```
@NoArgsConstructor
```

```
public class Usuario {
```

```
    @NotBlank(message = "Debe ingresar un nombre de usuario")
```

```
    private String nombreUsuario;
```

```
    @NotBlank(message = "Debe ingresar un valor correcto para la edad")
```

```
    @Pattern(regexp = "^[0-9]{1,2}$", message = "Edad incorrecta")
```

```
    private String edad;
```

```
    @AssertFalse(message = "Debe ser falso")
```

```
    private Boolean esfalso;
```

```
    @Pattern(regexp = "^[0-9]{4}-[0-9]{2}-[0-9]{2}$", message = "No es correcto el  
formato de la fecha")
```

```
private String fechanacimiento;  
  
}
```

Una vez se crean las anotaciones, se debe crear el controlador.

```
@Controller  
  
public class UserController {  
  
    @RequestMapping("/usuario")  
  
    public void create(@RequestBody @Validated Usuario usuario) {  
  
        System.out.println("Hola: " + usuario.getNombreUsuario());  
  
    }  
  
}
```

Una vez se realice el controlador, se puede realizar la prueba enviando una solicitud POST.

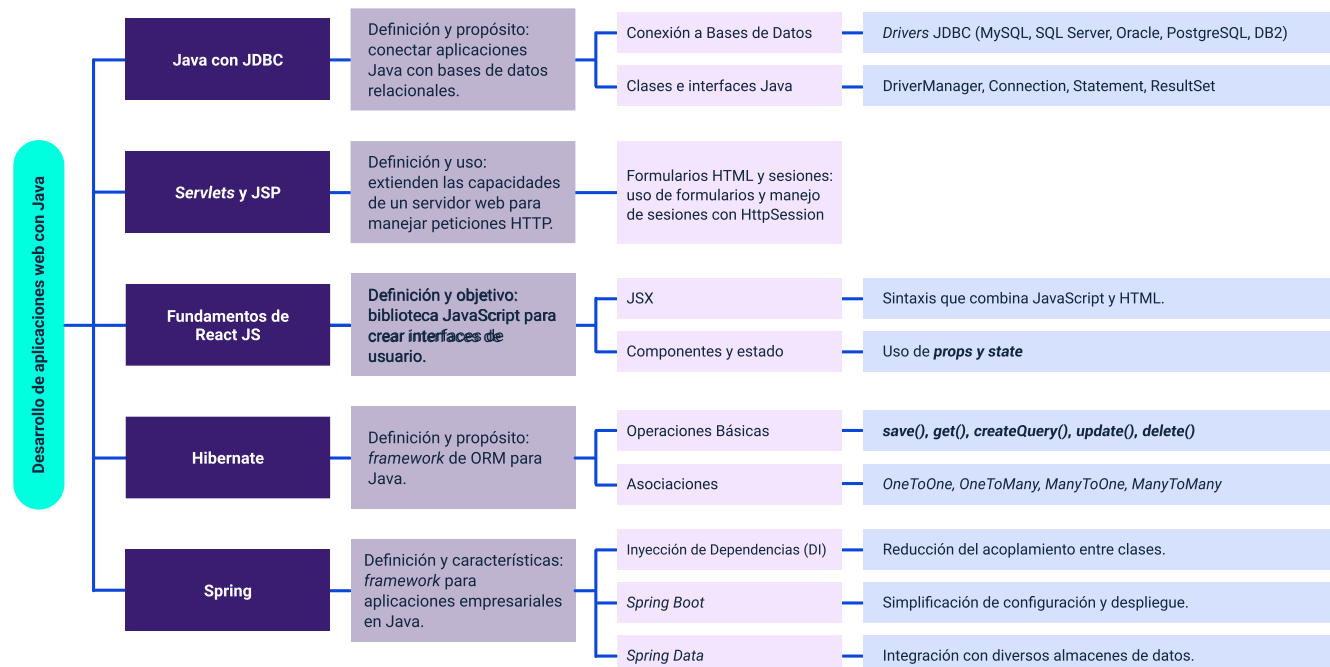
```
{"nombreUsuario": "Ana", "edad": 120, "esFalso": true, "nacimiento": "21010-  
21-12"}
```

La verificación que mostraría el resultado es:

- No es correcto el formato de la fecha.
- Debe ser falso.
- Debe ingresar un valor correcto para la edad.

Síntesis

A continuación, se presenta una síntesis de la temática estudiada en el componente formativo.



Material complementario

Tema	Referencia	Tipo de material	Enlace del recurso
Instalando React	Editor HTML gratuito en línea limpiador y convertidor. (2021, 27 de octubre). Editor HTML gratuito en línea limpiador y convertidor.	Página web	https://htmled.it/or/
Instalando React	Cubic Factory. (2021, 27 de octubre). Cubic Factory.	Página web	https://www.cubicfactory.com/jeditor/
Propiedades y estados de los componentes	CodeSandbox. (2021, 27 de octubre). CodeSandbox: Online Code Editor and IDE for Rapid Web Development.	Página web	https://codesandbox.io/
Crear un proyecto	React Playground. (2021, 27 de octubre). React Playground.	Página web	https://playcode.io/react
Hibernate	Hibernate Community Documentation. (s. f.). Capítulo 15. HQL: El lenguaje de consulta de Hibernate.	Página web	https://docs.jboss.org/hibernate/orm/3.5/reference/es-ES/html/queryhql.html

Glosario

API: conjunto de métodos ofrecidos por una librería que puede ser utilizado por otro software por medio de un conjunto de reglas establecidas.

CGI: interfaz de entrada común se refiere a un sistema de comunicación que le indica al servidor web cómo enviar y recibir datos de una aplicación de servidor a un cliente.

CSS3: hojas de estilo en cascada versión 3.

DOM: modelo de objetos de un documento HTML.

ES6: abreviación del estándar ECMA SCRIPT. Versión 6.

Framework: es un marco o esquema de trabajo que define un conjunto de utilidades implementadas para un contexto particular para ser reutilizadas por los desarrolladores y así facilitar y acelerar el desarrollo de ciertas actividades propias de la construcción de software.

Groovy: lenguaje de programación efectuado sobre la plataforma JAVA el cual permite realizar APIs de JAVA.

HQL: lenguaje de consulta parecido al SQL implementado por el framework Hibernate.

HTML: HyperText Markup Language, lenguaje de Marcas de Hipertexto.

JAVASCRIPT: lenguaje de programación.

JDBC: API para JAVA que permite el acceso y manipulación de bases de datos relacionales.

JDK: kit de Desarrollo de Java.

JPA: API de persistencia de JAVA.

JSX: sintaxis propuesta en React JS para escribir elementos de interfaz web.

Kotlin: lenguaje de programación de tipado estático, sencillo y fácil de entender y aprender.

NODE.js: es un servidor de aplicaciones.

NPM: es un manejador de paquetes de sistema operativo.

Plugin: programas pequeños que se anexan a otros para ofrecer alguna funcionalidad nueva.

SPA: tipo de aplicación Web.

URL: localizador uniforme de recursos, facilita la recuperación de información determinada en internet, se escribe en el navegador para ingresar a una página determinada.

Referencias bibliográficas

Berzal, F., Cubero, J., & Cortijo, F. (2007). Desarrollo profesional de aplicaciones web con Asp. net.

https://books.google.es/books?hl=es&lr=&id=J1d_9l6zIAIC&oi=fnd&pg=PA3&dq=servlests+y+jsp+fernando+berzal&ots=GSR9oMYn8M&sig=YaCnuz6ed8E_kEo_G1CPHH-KjjU

Capítulo 15. HQL: El lenguaje de consulta de Hibernate. (s. f.). Hibernate Community Documentation. <https://docs.jboss.org/hibernate/orm/3.5/reference/es-ES/html/queryhql.html>

Gallagher, J. (2021, 4 de mayo). Los lenguajes de programación más populares. Career Karma. <https://careerkarma.com/blog/top-programming-languages-2021/>

Gómez, M., & Cervantes, J. (2017). Introducción a la Programación Web con Java: JSP y Servlets JavaServer Faces.

Johnson, R. (2004). Expert one-on-one J2EE design and development. John Wiley & Sons.

Martínez, P. E. (2015). Hibernate. Persistencia de objetos en JEE (1ª ed.). Ra-Ma.

Node. (2021). Node JS Server. <https://nodejs.org/en/>

React JS. (2021). React - A JavaScript library for building user interfaces. <https://reactjs.org/>

Spring. (2021). Spring makes Java simple. <https://spring.io>

Statistics Times. (2021). Principales idiomas informáticos. STATISTICSTIMES. <https://statisticstimes.com/tech/top-computer-languages.php>

Créditos

Nombre	Cargo	Centro de Formación y Regional
Milady Tatiana Villamil Castellanos	Responsable del Ecosistema	Dirección General
Olga Constanza Bermúdez Jaimes	Responsable de Línea de Producción	Centro de Servicios de Salud - Regional Antioquia
Carlos Hernán Muñoz Carvajal	Experto Temático	Centro de Teleinformática y Producción Industrial - Regional Cauca
Zulema Yidney León Escobar	Experta Temática	Centro de Teleinformática y Producción Industrial - Regional Cauca
Paola Alexandra Moya Peralta	Evaluadora Instruccional	Centro de Servicios de Salud - Regional Antioquia
Juan Daniel Polanco Muñoz	Diseñador de Contenidos Digitales	Centro de Servicios de Salud - Regional Antioquia
Jhon Jairo Urueta Álvarez	Desarrollador Fullstack	Centro de Servicios de Salud - Regional Antioquia
Daniela Muñoz Bedoya	Animador y Productor Multimedia	Centro de Servicios de Salud - Regional Antioquia
Edgar Mauricio Cortés García	Actividad Didáctica	Centro de Servicios de Salud - Regional Antioquia
Luis Gabriel Urueta Alvarez	Validador de Recursos Educativos Digitales	Centro de Servicios de Salud - Regional Antioquia
Margarita Marcela Medrano Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia
Daniel Ricardo Mutis Gómez	Evaluador para Contenidos Inclusivos y Accesibles	Centro de Servicios de Salud - Regional Antioquia